

Exercise – 13 Stress Testing a Linux EC2 Instance (CPU Load Test)

Objective:

To generate high CPU load on an Amazon EC2 Linux instance using the stress-ng tool and observe CPU utilization in CloudWatch.

Step 1 — Launch a Linux EC2 Instance

1. Log in to the AWS Console.
2. Go to **EC2 → Instances → Launch Instance**.
3. Name: **StressTest-EC2**
4. AMI: **Amazon Linux 2023 (Free Tier eligible)**
5. Instance type: **t2.micro / t3.micro**
6. Key pair: Select or create a .pem key.
7. Security Group:
 - Allow **SSH (22)** from My IP.
 - Allow **HTTP (80)** from anywhere (optional).
8. Launch the instance.

Step 2 — Connect to the EC2 Instance

Use Windows PowerShell

Navigate to the folder where pem file is located and type the following command:

```
ssh -i "your-key.pem" ec2-user@<public-ip>
```

Step 3 — Install Apache (Optional, to verify the instance is working)

```
sudo yum install httpd -y
```

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

Create a test web page:

```
echo "<h1>EC2 Stress Test Demo — $(hostname)</h1>" | sudo tee  
/var/www/html/index.html
```

STEP 4 — Install and Run the Stress Testing Tool (stress-ng)

(For Amazon Linux 2023 EC2 Instances)

Move to the ec2-user home directory

(It is recommended to run stress-ng from the home folder)

```
cd ~
```

Install stress-ng

Amazon Linux 2023 includes stress-ng directly through dnf, so install it using:

```
sudo dnf install stress-ng -y
```

Run a CPU Stress Test

Generate high CPU load using 4 CPU workers for 2 minutes:

```
stress-ng --cpu 4 --timeout 120
```

Expected Result

- Terminal becomes busy during the 120-second load test
- After completion, you will see:

successful run completed in 120.02s

- CPU usage will spike to 90–100% (you can check using top)

Step 5 — Run Stress Test (CPU Load Generation)

Move to home directory to avoid permission issues:

```
cd ~
```

Run CPU stress for 2 minutes using 4 CPU workers:

```
stress-ng --cpu 4 --timeout 120
```

You should see messages like:

```
stress-ng: info: dispatching hogs: 4 cpu
```

```
stress-ng: info: successful run completed in 120.02s
```

This will increase CPU usage to ~100% for the duration.

6A. Create Launch Template

- EC2 → Launch Templates → Create
- Name: WebServer-LT
- AMI: WebServer-AMI
- Instance type: t3.micro
- Security Group: allow HTTP + SSH
- Create

EC2 > Launch templates > Create launch template

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - *required*

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

Max 255 chars

Auto Scaling guidance | [Info](#)
 Select this if you intend to use this template with EC2 Auto Scaling

☒ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

► **Template tags**

► **Source template**

▼ Summary

Software Image (AMI)
 WebServer-AMI
 ami-0920f39c3335fd68e

Virtual server type (instance type)
 t3.micro

Firewall (security group)
 launch-wizard-3

Storage (volumes)
 1 volume(s) - 8 GiB

Cancel **Create launch template**

The image displays two screenshots of the AWS Management Console, specifically the 'Create launch template' page. The top screenshot shows the 'Instance type' section with 't3.micro' selected. The bottom screenshot shows the 'Network settings' section with 'launch-wizard-3' selected as the security group.

Top Screenshot: Instance type section

- Instance type:** t3.micro (Free tier eligible)
- Family: t3, 2 vCPU, 1 GiB Memory, Current generation: true
- On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour
- On-Demand SUSE base pricing: 0.0104 USD per Hour
- On-Demand Linux base pricing: 0.0104 USD per Hour
- On-Demand RHEL base pricing: 0.0392 USD per Hour
- On-Demand Windows base pricing: 0.0196 USD per Hour
- Additional costs apply for AMIs with pre-installed software**
- Advanced:** All generations (selected), Compare instance types

Summary section (top):

- Software Image (AMI):** WebServer-AMI (ami-0920f39e3335fd68e)
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** launch-wizard-3
- Storage (volumes):** 1 volume(s) - 8 GiB
- Buttons:** Cancel, Create launch template

Bottom Screenshot: Network settings section

- Subnet:** Don't include in launch template (selected), Create new subnet
- Availability Zone:** Don't include in launch template (selected), Enable additional zones
- Firewall (security groups):** Select existing security group (selected), Create security group
- Security groups:** Select security groups (selected), Compare security group rules
- Selected security group:** launch-wizard-3 sg-0e25a0e1761f68109 (VPC: vpc-0e06f443318ef0d95)
- Advanced network configuration:** (link)

Summary section (bottom):

- Software Image (AMI):** WebServer-AMI (ami-0920f39e3335fd68e)
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** launch-wizard-3
- Storage (volumes):** 1 volume(s) - 8 GiB
- Buttons:** Cancel, Create launch template

Storage (volumes) section (bottom):

- Storage (volumes):** EBS Volumes (selected), Hide details

6B. Create Auto Scaling Group

- Go to Auto Scaling → Create Auto Scaling Group
- Name: WebApp-ASG
- Select Launch Template: WebServer-LT
- Choose VPC + two public subnets
- Load Balancing:
 - Attach to existing ALB
 - Choose WebApp-TG
- Group size:
 - Desired: 1
 - Min: 1
 - Max: 3
- Create

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1: Choose launch template
Step 2: Choose instance launch options
Step 3 - optional: **Integrate with other services**
Step 4 - optional: Configure group size and scaling
Step 5 - optional: Add notifications
Step 6 - optional: Add tags
Step 7: Review

Integrate with other services - optional [Info](#)

Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.

Load balancing [Info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Select Load balancing options

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☒ **Attach to an existing load balancer**
Choose from your existing load balancers.

☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer

Select the load balancers to attach

☒ **Choose from your load balancer target groups**
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

Webapp-tg | HTTP
Application Load Balancer: WebApp-ALB

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 4 - optional: **Configure group size and scaling**
Step 5 - optional: Add notifications
Step 6 - optional: Add tags
Step 7: Review

Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity
Specify your group size.

1

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity
1
Equal or less than desired capacity

Max desired capacity
3
Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy [Info](#)
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☒ **No scaling policies**
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☐ Target tracking scaling policy
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Instance maintenance policy [Info](#)

STEP 7 — Configure Auto Scaling Policies

Go to ASG → Automatic Scaling → Add Policy

Scale-Out Policy

- Metric: CPU Utilization
- Rule: > 60% for 2 minutes
- Action: Add +1 instance

Scale-In Policy

- Rule: < 20% for 5 minutes
- Action: Remove 1 instance

STEP 8 — Stress Test to Trigger Scaling

SSH into your running instance:

Install stress:

```
sudo amazon-linux-extras install epel -y
```

```
sudo yum install stress -y
```

Run CPU load:

```
stress --cpu 90 --timeout 300
```

Within 2–3 minutes:

- CPU becomes high
- ASG launches second instance
- Target Group shows both instances as healthy

STEP 9 — Test Load Balancing

Open ALB DNS Name in browser: `http://<alb-dns-name>`

Refresh many times — you will see:

Hello from Instance 1 — ip-XX-XX...

Hello from Instance 2 — ip-YY-YY...

- Confirms ALB working
- Confirms auto scaling working

After stress test ends → ASG scales back to 1 instance.

DATE: 26-11-25

EXERCISE - 17: Flask Web App with Registration & Login Using AWS RDS (MySQL)

(Full Deployment on AWS EC2 + AWS RDS)

To design and deploy a Python Flask web application on an AWS EC2 instance that allows:

1. User Registration – store Name and Password in AWS RDS MySQL database.
2. User Login – validate user credentials from the RDS database.
3. Welcome Page – display a message on successful login.

This exercise demonstrates a complete cloud-based web application workflow using Flask + MySQL (RDS).

System Architecture

Flow: Browser → Flask App (on EC2) → MySQL Database (RDS)

- Frontend: HTML templates (registration form, login form, welcome page)
- Backend: Python Flask application running on EC2
- Database: AWS RDS – MySQL engine
- Hosting: Linux EC2 instance
- Web Server: Flask's built-in development server (Apache/NGINX not used in this exercise)

Folder Structure (on EC2 instance)

Create the following structure inside the EC2 instance:

FlaskLoginApp/

```
|— app.py
|— db_config.py
└— templates/
    |— register.html
    |— login.html
    |— success.html
    |— error.html
    └— welcome.html
```

From EC2 Create Database and Table in RDS MySQL

Where:

The SQL commands are executed from the EC2 instance, by connecting to the RDS MySQL database using the MySQL client.

When:

After the RDS MySQL instance status becomes “Available” and after allowing EC2 security group access in RDS inbound rules.

Create:

- A database: flaskdb
- A table: users

SQL: Create Database and Table

```
CREATE DATABASE flaskdb;
```

```
USE flaskdb;
```

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  password VARCHAR(50) NOT NULL
);
```

- id – unique identifier for each user (auto-increment primary key).
- name – username (cannot be NULL).
- password – password (cannot be NULL).

Note: For learning purposes, we store plain text. In real systems, passwords must be hashed.

Flask Application Code (Backend)

6.1 db_config.py

This file stores the **RDS MySQL connection details**.

```
# db_config.py
```

```
db_config = {
    'host': 'your-rds-endpoint.amazonaws.com',
    # e.g., flaskdb-instance.xxxxxx.ap-south-1.rds.amazonaws.com
    'user': 'admin',                # master username created in RDS
    'password': 'YourPasswordHere', # master password created in RDS
    'database': 'flaskdb'          # database name created in MySQL
}
```

Replace host, user, and password with the actual values from the RDS instance.

6.2 app.py

This is the **main Flask application**.

app.py

```
from flask import Flask, request, render_template
import mysql.connector
from db_config import db_config

app = Flask(__name__)

# Function to create a new database connection
def get_connection():
    return mysql.connector.connect(**db_config)

@app.route('/')
def home():
    return render_template('register.html')

@app.route('/register', methods=['POST'])
def register():
    name = request.form['uname']
    password = request.form['pwd']

    conn = get_connection()
    cur = conn.cursor()

    sql = "INSERT INTO users (name, password) VALUES (%s, %s)"
    cur.execute(sql, (name, password))
    conn.commit()

    cur.close()
    conn.close()

    # Use HTML template for success page
    return render_template('success.html')

@app.route('/login')
def login():
    return render_template('login.html')
```



```

@app.route('/validate', methods=['POST'])
def validate():
    name = request.form['uname']
    password = request.form['pwd']

    conn = get_connection()
    cur = conn.cursor()

    sql = "SELECT * FROM users WHERE name = %s AND password = %s"
    cur.execute(sql, (name, password))
    user = cur.fetchone()

    cur.close()
    conn.close()

    if user:
        # Successful login -> welcome page
        return render_template('welcome.html', username=name)
    else:
        # Invalid login -> error page
        return render_template('error.html')

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5000, debug=True)

```

7. HTML Templates (Frontend)

All HTML files go inside the templates/ folder.

7.1 templates/register.html

```

<!DOCTYPE html>
<html>
<head>
    <title>User Registration</title>
</head>
<body>
    <h2>User Registration</h2>

    <form action="/register" method="post">
        <label>Name:</label>
        <input type="text" name="uname" required><br><br>

        <label>Password:</label>
        <input type="password" name="pwd" required><br><br>

        <button type="submit">Register</button>
    </form>

```

```
<br>
<a href="/login">Already registered? Click here to Login</a>
</body>
</html>
```

7.2 templates/login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>User Login</title>
</head>
<body>
  <h2>User Login</h2>

  <form action="/validate" method="post">
    <label>Name:</label>
    <input type="text" name="uname" required><br><br>

    <label>Password:</label>
    <input type="password" name="pwd" required><br><br>

    <button type="submit">Login</button>
  </form>

  <br>
  <a href="/">New user? Click here to Register</a>
</body>
</html>
```

7.3 templates/welcome.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Welcome</title>
</head>
<body>
  <h2>Welcome, {{ username }}!</h2>
  <p>You have successfully logged in.</p>
</body>
</html>
```

7.4 templates/success.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Successful</title>
</head>
<body>
```

```
<h3>Registration Successful!</h3>
<a href="/login">Click here to Login</a>
</body>
</html>
```

7.5 templates/error.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login Error</title>
</head>
<body>
  <h3>Invalid Username or Password</h3>
  <a href="/login">Try Again</a>
</body>
</html>
```

Deployment Steps –

PHASE 1 – Launch EC2 Linux Instance

- Open AWS Management Console → EC2 → Launch instance.
- Configuration:
 - Name: FlaskLoginServer
 - AMI: Amazon Linux 2023 (Free Tier eligible)
 - Instance type: t3.micro (Free Tier)
 - Key pair: create or select existing
 - Network: Default VPC and any public subnet
 - Auto-assign Public IP: Enable
- Security Group for EC2 – Inbound rules:

Type	Port	Source	Purpose
SSH	22	My IP	To connect via SSH/EC2 Connect
Custom TCP	5000	0.0.0.0/0	To test Flask app in browser

- Outbound: Allow all traffic.
- Click Launch instance.

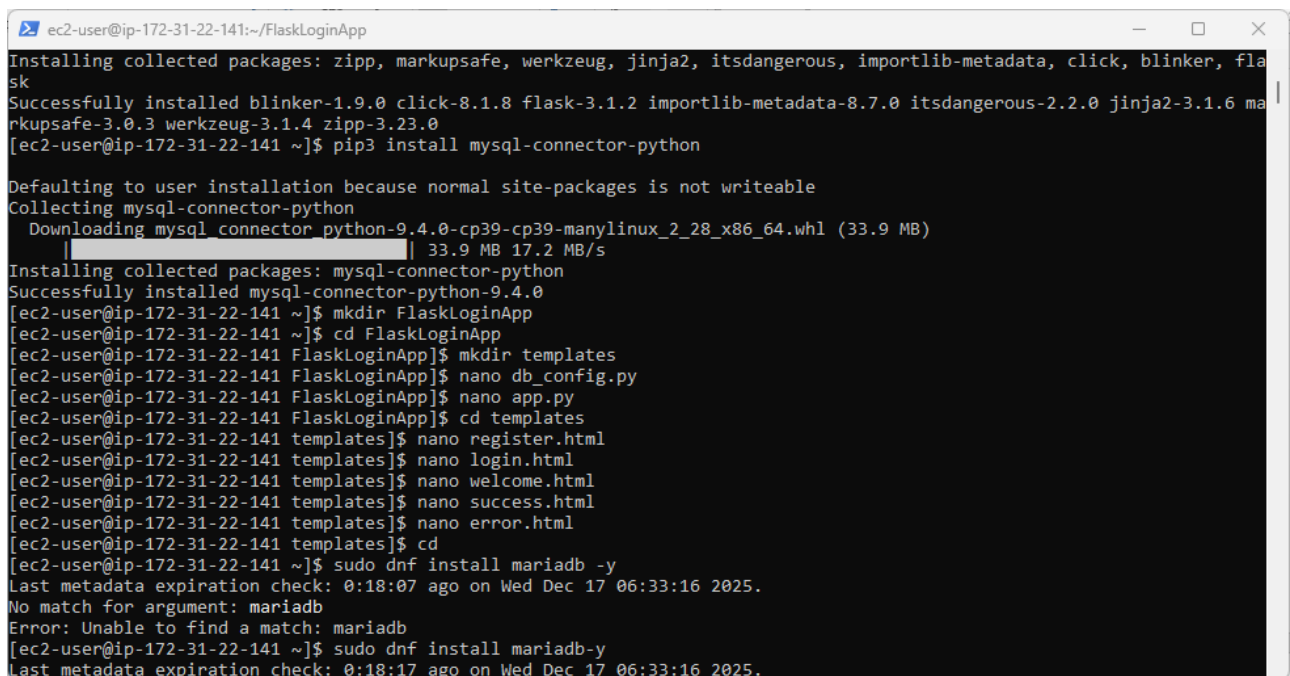
PHASE 2 – Connect to EC2 and Install Dependencies

- Go to EC2 → Instances → Select instance → Connect.
- Choose EC2 Instance Connect (browser-based SSH) → Connect.
Or open PowerShell on windows and connect using ssh command
- Update packages:
 sudo yum update -y
- Check Python version (Amazon Linux 2023 has Python 3):
 python3 --version
- Install pip (if not already installed):
 sudo dnf install python3-pip -y
- Install Flask and MySQL Connector for Python:
 pip3 install flask
 pip3 install mysql-connector-python

PHASE 3 – Create Flask Project Structure on EC2

- Create project folder:
 mkdir FlaskLoginApp
 cd FlaskLoginApp
- Create templates folder:
 mkdir templates
- Create the Python and HTML files using nano:
 nano db_config.py → paste code for db_config.py
 nano app.py → paste code for app.py
 nano templates/register.html → paste register page
 nano templates/login.html → paste login page
 nano templates/welcome.html → paste welcome page

Save each file using: CTRL + O → Enter → CTRL + X



```
ec2-user@ip-172-31-22-141:~/FlaskLoginApp
Installing collected packages: zipp, markupsafe, werkzeug, jinja2, itsdangerous, importlib-metadata, click, blinker, flask
Successfully installed blinker-1.9.0 click-8.1.8 flask-3.1.2 importlib-metadata-8.7.0 itsdangerous-2.2.0 jinja2-3.1.6 ma
rkupsafe-3.0.3 werkzeug-3.1.4 zipp-3.23.0
[ec2-user@ip-172-31-22-141 ~]$ pip3 install mysql-connector-python

Defaulting to user installation because normal site-packages is not writeable
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.4.0-cp39-cp39-manylinux_2_28_x86_64.whl (33.9 MB)
    | 33.9 MB 17.2 MB/s
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.4.0
[ec2-user@ip-172-31-22-141 ~]$ mkdir FlaskLoginApp
[ec2-user@ip-172-31-22-141 ~]$ cd FlaskLoginApp
[ec2-user@ip-172-31-22-141 FlaskLoginApp]$ mkdir templates
[ec2-user@ip-172-31-22-141 FlaskLoginApp]$ nano db_config.py
[ec2-user@ip-172-31-22-141 FlaskLoginApp]$ nano app.py
[ec2-user@ip-172-31-22-141 FlaskLoginApp]$ cd templates
[ec2-user@ip-172-31-22-141 templates]$ nano register.html
[ec2-user@ip-172-31-22-141 templates]$ nano login.html
[ec2-user@ip-172-31-22-141 templates]$ nano welcome.html
[ec2-user@ip-172-31-22-141 templates]$ nano success.html
[ec2-user@ip-172-31-22-141 templates]$ nano error.html
[ec2-user@ip-172-31-22-141 templates]$ cd
[ec2-user@ip-172-31-22-141 ~]$ sudo dnf install mariadb -y
Last metadata expiration check: 0:18:07 ago on Wed Dec 17 06:33:16 2025.
No match for argument: mariadb
Error: Unable to find a match: mariadb
[ec2-user@ip-172-31-22-141 ~]$ sudo dnf install mariadb-y
Last metadata expiration check: 0:18:17 ago on Wed Dec 17 06:33:16 2025.
```

PHASE 4 – Create RDS MySQL Instance

- Go to AWS Console → RDS → Databases → Create database.
 - Engine type: MySQL
 - Templates: Free tier

Create database [Info](#)

Choose a database creation method

☒ **Full configuration**
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ **Easy create**
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type [Info](#)

☒ **Aurora (MySQL Compatible)**

☐ Aurora (PostgreSQL Compatible)

☐ MySQL

☐ PostgreSQL

☐ MariaDB

☐ Oracle

☐ Microsoft SQL Server

☐ IBM Db2

Engine version

- DB instance identifier: flaskdb-instance
- Master username: admin (example)

Settings

DB cluster identifier [Info](#)
Type a name for your DB cluster. The name must be unique across all DB clusters owned by your AWS account in the current AWS Region.

flaskdb-instance

The DB cluster identifier is case-insensitive, but is stored as all lowercase (as in "mydbcluster"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

▼ Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.

admin

1 to 32 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☒ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☐ **Self managed**
Create your own password or have RDS create a password that you manage.

☒ If you manage the master user credentials in AWS Secrets Manager, additional charges apply. See [AWS Secrets Manager pricing](#). Additionally, some RDS features aren't supported. See [limitations here](#).

Select the encryption key [Info](#)
You can encrypt using the KMS key that Secrets Manager creates or a customer managed KMS key that you create.

- Master password: <your-password>

▼ Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.
admin

1 to 32 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - most secure
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed
Create your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength **Very weak**
Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' * @

Confirm master password [Info](#)

Cluster storage configuration [Info](#)
Choose the storage configuration for the Aurora DB cluster that best fits your application's price predictability and price performance needs.

- DB instance class: db.t3.micro (Free tier)
- Storage: e.g. 20 GB (Free tier)
- Connectivity:
 - Connect to an EC2 compute resource → YES
 - Select the same EC2 instance (or its security group)

Connectivity [Info](#)

Compute resource
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☐ Don't connect to an EC2 compute resource
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☒ Connect to an EC2 compute resource
Set up a connection to an EC2 compute resource for this database.

EC2 instance [Info](#)
Choose the EC2 instance to add as the compute resource for this database. A VPC security group is added to this EC2 instance. A VPC security group is also added to the database with an inbound rule that allows the EC2 instance to access the database.

i-OaOed5f1afa99348e
FlaskLoginServer

Some VPC settings can't be changed when a compute resource is added
Adding an EC2 compute resource automatically selects the VPC, DB subnet group, and public access settings for this database. To allow the EC2 instance to access the database, a VPC security group rds-ec2-X is added to the database and another called ec2-rds-X to the EC2 instance. You can remove the new security group for the database only by removing the compute resource.

Network type [Info](#)
To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

☒ IPv4
Your resources can communicate only over the IPv4 addressing protocol.

☐ Dual-stack mode
Your resources can communicate over IPv4, IPv6, or both.

Virtual private cloud (VPC) [Info](#)
Choose the VPC. The VPC defines the virtual networking environment for this DB cluster.

- (AWS automatically performs the following:
 - Selects the same VPC as EC2
 - Creates/associates an RDS security group

- Adds inbound rule: MySQL (3306) from EC2 security group)
- Public access: No
- Click Create database.
Wait until the status becomes “Available”.

PHASE 5 – Verify RDS Security Group (Auto-configured)

- Go to AWS Console → RDS → Databases → flaskdb-instance.
- Open Connectivity & security section.
- Under VPC security groups, click the linked security group name.
- In Inbound rules, verify that the following rule exists:

Type	Port	Source
MySQL / Aurora	3306	EC2 Security Group

0. If the rule exists, EC2 to RDS connectivity is correctly configured.

PHASE 6 – Create Database and Table in RDS from EC2

- From the EC2 terminal, install MySQL client (MariaDB client):
sudo dnf install mariadb105 -y
if this fails, try: sudo dnf install mariadb -y
- Connect to the RDS MySQL instance from EC2:
mysql -h <RDS-endpoint> -u admin -p
Example: mysql -h flaskdb-instance.xxxxxx.ap-south-1.rds.amazonaws.com -u admin -p
- Inside the MySQL prompt, run following commands:

```
CREATE DATABASE flaskdb;
```

```
USE flaskdb;
```

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  password VARCHAR(50) NOT NULL
);
```

- Exit MySQL: exit;

```
Select ec2-user@ip-172-31-22-141:~/FlaskLoginApp
perl-Sys-Hostname-1.23-477.amzn2023.0.7.x86_64

Complete!
[ec2-user@ip-172-31-22-141 ~]$ mysql -h flaskdb-instance.cluster-cliiikagwm2d2.ap-southeast-2.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 60
Server version: 8.0.39 8bc99e28

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> CREATE DATABASE flaskdb;
Query OK, 1 row affected (0.002 sec)

MySQL [(none)]> USE flaskdb;
Database changed
MySQL [flaskdb]> CREATE TABLE users (
->   id INT AUTO_INCREMENT PRIMARY KEY,
->   name VARCHAR(50) NOT NULL,
->   password VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.015 sec)

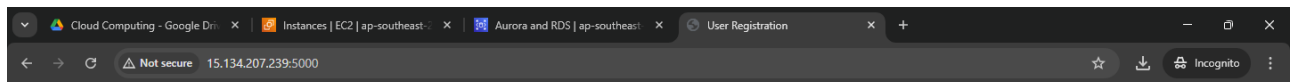
MySQL [flaskdb]> exit
Bye
[ec2-user@ip-172-31-22-141 ~]$ cd FlaskLoginApp/
[ec2-user@ip-172-31-22-141 FlaskLoginApp]$ nano db_config.py
```

PHASE 7 – Configure Flask to Use RDS and Run Application

- Open db_config.py on EC2 and update with **correct RDS details**:
nano db_config.py
Ensure:
db_config = {
 'host': 'your-rds-endpoint.amazonaws.com',
 'user': 'admin',
 'password': 'YourPasswordHere',
 'database': 'flaskdb'
}
Save and exit.
- Start the Flask app:
cd ~/FlaskLoginApp
python3 app.py
You should see: * Running on http://0.0.0.0:5000

PHASE 8 – Test the Application from Browser

- In your local system browser, open: http://<EC2-public-IP>:5000
Example: http://13.232.122.81:5000
You should see: Registration Page (default route /)



User Registration

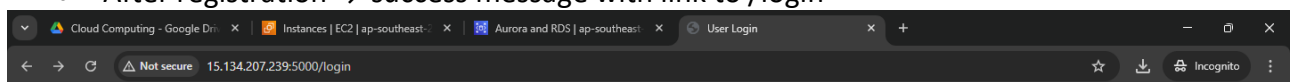
Name:

Password:

[Already registered? Click here to Login](#)

Activate Windows
Go to Settings to activate Windows.

- After registration → success message with link to /login



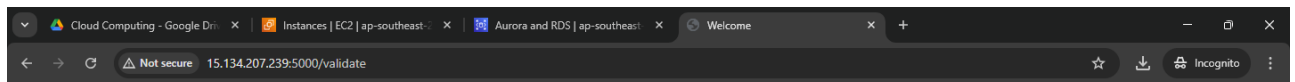
User Login

Name:

Password:

[New user? Click here to Register](#)

Activate Windows
Go to Settings to activate Windows.



Welcome, abc!

You have successfully logged in.

Activate Windows
Go to Settings to activate Windows.

- Login Page → if correct credentials → Welcome Page
Check that data is stored in flaskdb.users table in RDS. - `SELECT * FROM users;`

Expected Output

1. Registration Page
 - User enters Name and Password.
 - On submit, data is inserted into users table in RDS MySQL.
2. Login Page
 - User enters same Name and Password.
 - Flask executes `SELECT` query on users table to check match.
3. Welcome Page
 - For valid credentials:
“Welcome, <username>! You have successfully logged in.”
 - For invalid credentials:
“Invalid Username or Password” message with link to try again.

Result

A **fully working cloud-based registration and login system** was successfully deployed using:

- **Flask** as the backend web framework
- **HTML** templates for frontend pages
- **MySQL** database hosted on **AWS RDS**
- **AWS EC2** instance as the application server

This exercise demonstrates a **complete end-to-end mini project** on AWS using **Flask + MySQL (RDS)**.

Sample Viva Questions

1. What is **Amazon RDS**? Why do we use it instead of installing MySQL directly on EC2?
2. What is the difference between **EC2** and **RDS**?
3. How does **Flask** connect to **MySQL** in this exercise?
4. What is a **POST request**? Where is it used in this application?
5. Why must the **EC2 security group** be allowed inside **RDS inbound rules**?

AWS RDS databases are private by default and do not allow any incoming connections unless explicitly permitted by security group rules.

Since the Flask application runs on EC2, it must be allowed to connect to the RDS MySQL database.

Therefore, the EC2 security group must be added to the RDS inbound rules to allow:

- Port: 3306 (MySQL)
- Source: EC2 Security Group

This ensures that only the EC2 instance running the application can access the database.

6. What is the purpose of db_config.py?
 7. What is **SQL Injection**? How does using parameterized queries (%s) help prevent it?
- **SQL Injection:**
A technique where malicious SQL code is entered as user input.
In this application, it is prevented using parameterized queries (%s).
 - What is the difference between **Registration workflow** and **Login workflow** in this app?
 - Why do we use host='0.0.0.0' in app.run() on EC2?
 - By default, Flask runs on 127.0.0.1 (localhost), which means the application is accessible **only from inside the EC2 instance**.

When we set: app.run(host='0.0.0.0', port=5000)

it tells Flask to **listen on all network interfaces** of the EC2 instance.

This allows:

- Access from the **browser on our local system**
- Access using the **EC2 public IP address**

What happens if we don't use 0.0.0.0?

- Flask listens only on localhost
- Application works **inside EC2**
- Browser access using http://<EC2-public-IP>:5000 fails

In this exercise

- Flask runs on EC2
- Users access it from outside EC2

- Therefore, Flask must listen on all interfaces using 0.0.0.0

We use host='0.0.0.0' so that the Flask application running on EC2 is accessible from external systems using the EC2 public IP.

8. Why is debug=True not recommended in production?

Exercise - 18: Creating and Operating a NoSQL Key-Value Database using Amazon DynamoDB

To create an Amazon DynamoDB table and perform CRUD operations (Create, Read, Update, Delete) using the AWS Management Console, and understand Partition Key, Sort Key, Query vs Scan, and table cleanup.

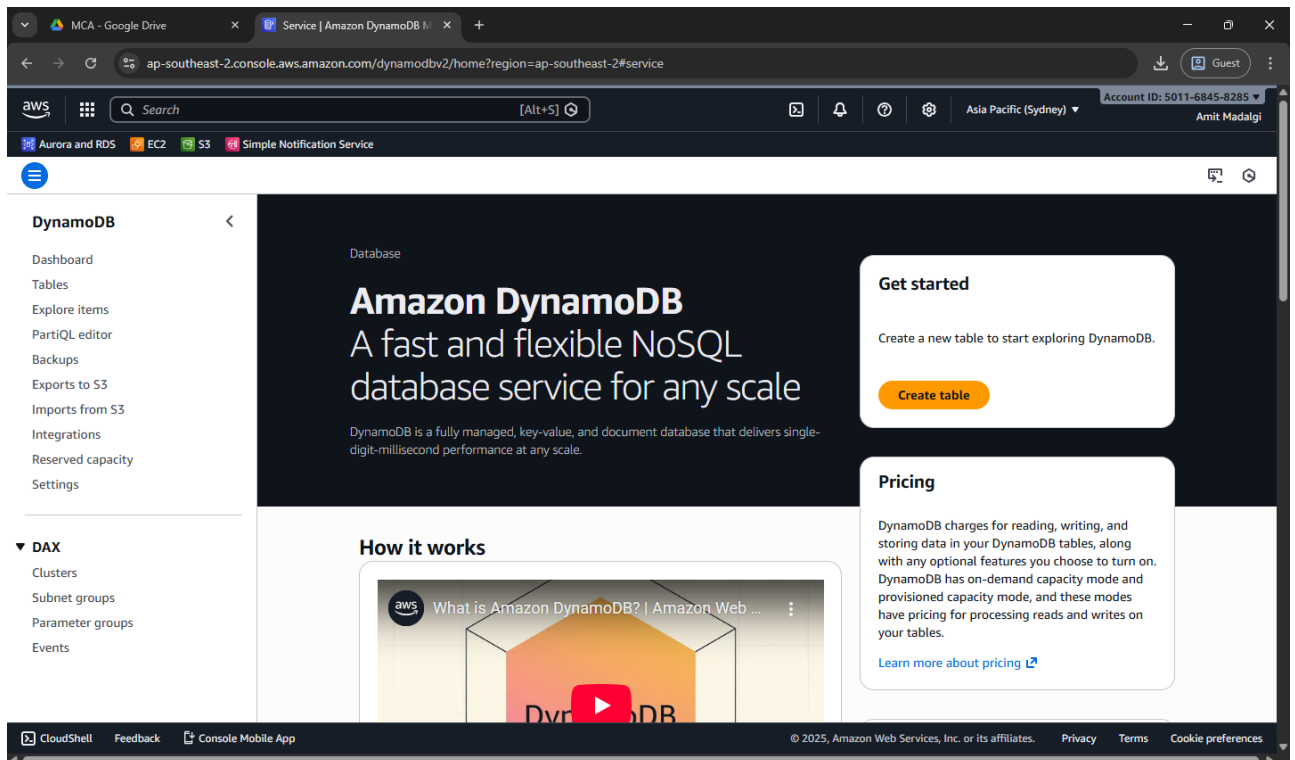
Phase 1: Create DynamoDB Table

Create a DynamoDB table to store Student Records.

Store items like: USN, Name, Semester, Course, Attendance, IA1Marks

Step 1: Open DynamoDB

- Login to AWS Console
- Search → type DynamoDB → open Amazon DynamoDB



Step 2: Create a Table

- Left menu → Tables
- Click Create table
- Fill details:

Table details

- Table name: MCA_StudentLabInternals
- Partition key (PK): USN (Type: String)
- Sort key (SK): CourseCode (Type: String)
(Enable sort key option by setting sort key)

Why this design?

- One student can have multiple courses → Sort key separates records per course.
- This creates a composite primary key (PK + SK).

The screenshot shows the AWS Management Console interface for creating a new DynamoDB table. The browser address bar indicates the URL: `ap-southeast-2.console.aws.amazon.com/dynamodbv2/home?region=ap-southeast-2#create-table`. The console header shows the AWS logo, a search bar, and navigation links for various services like Aurora and RDS, EC2, S3, and Simple Notification Service. The user is logged in as 'Amit Madalgi' with account ID '5011-6845-8285' in the 'Asia Pacific (Sydney)' region.

The 'Create table' page is divided into two main sections: 'Table details' and 'Table settings'.

Table details (Info icon):

- Table name:** A text input field contains 'MCA_StudentLabInternals'. Below it, a note states: 'Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.)'.
- Partition key:** A text input field contains 'USN', and a dropdown menu shows 'String'. A note below states: '1 to 255 characters and case sensitive'.
- Sort key - optional:** A text input field contains 'CourseCode', and a dropdown menu shows 'String'. A note below states: '1 to 255 characters and case sensitive'.

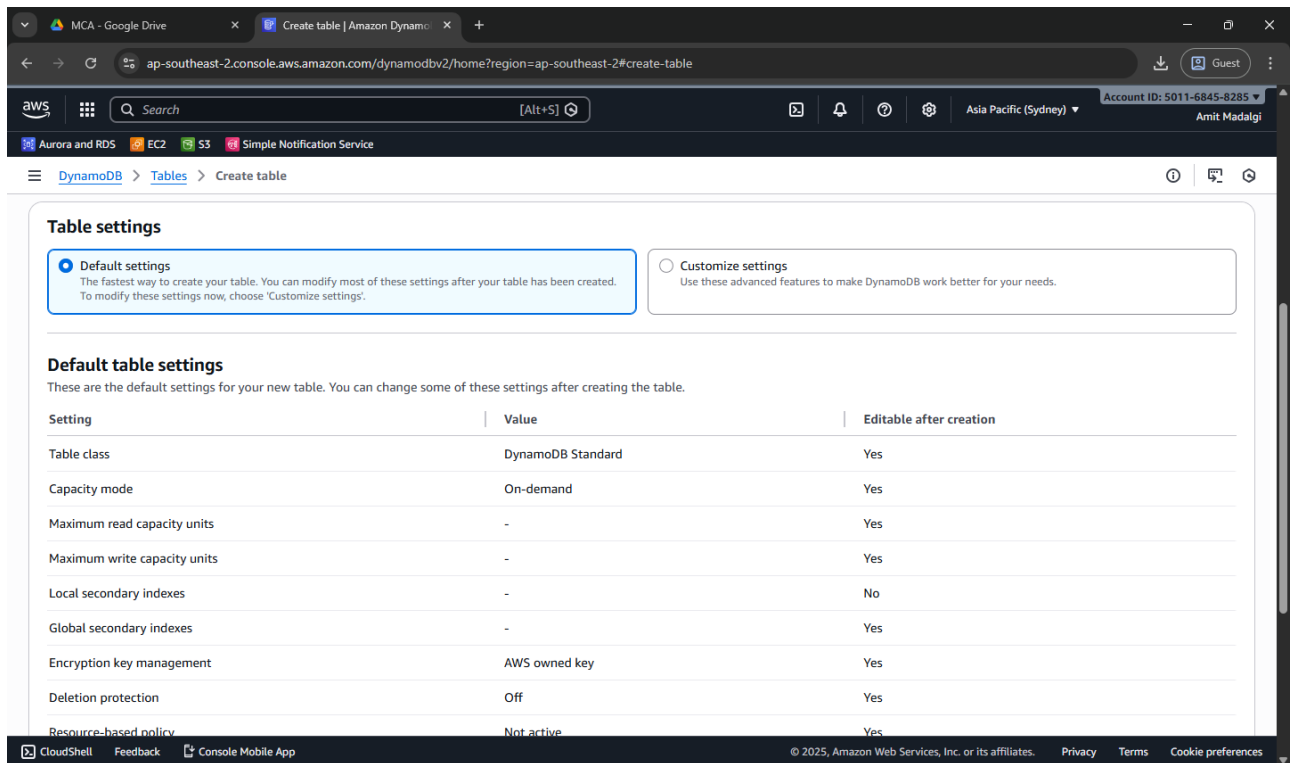
Table settings:

- Default settings:** Selected with a radio button. A note below states: 'The fastest way to create your table. You can modify most of these settings after your table has been created.'
- Customize settings:** Unselected with a radio button. A note below states: 'Use these advanced features to make DynamoDB work better for your needs.'

The footer of the console shows links for 'CloudShell', 'Feedback', and 'Console Mobile App', along with copyright information for Amazon Web Services, Inc. and links for 'Privacy', 'Terms', and 'Cookie preferences'.

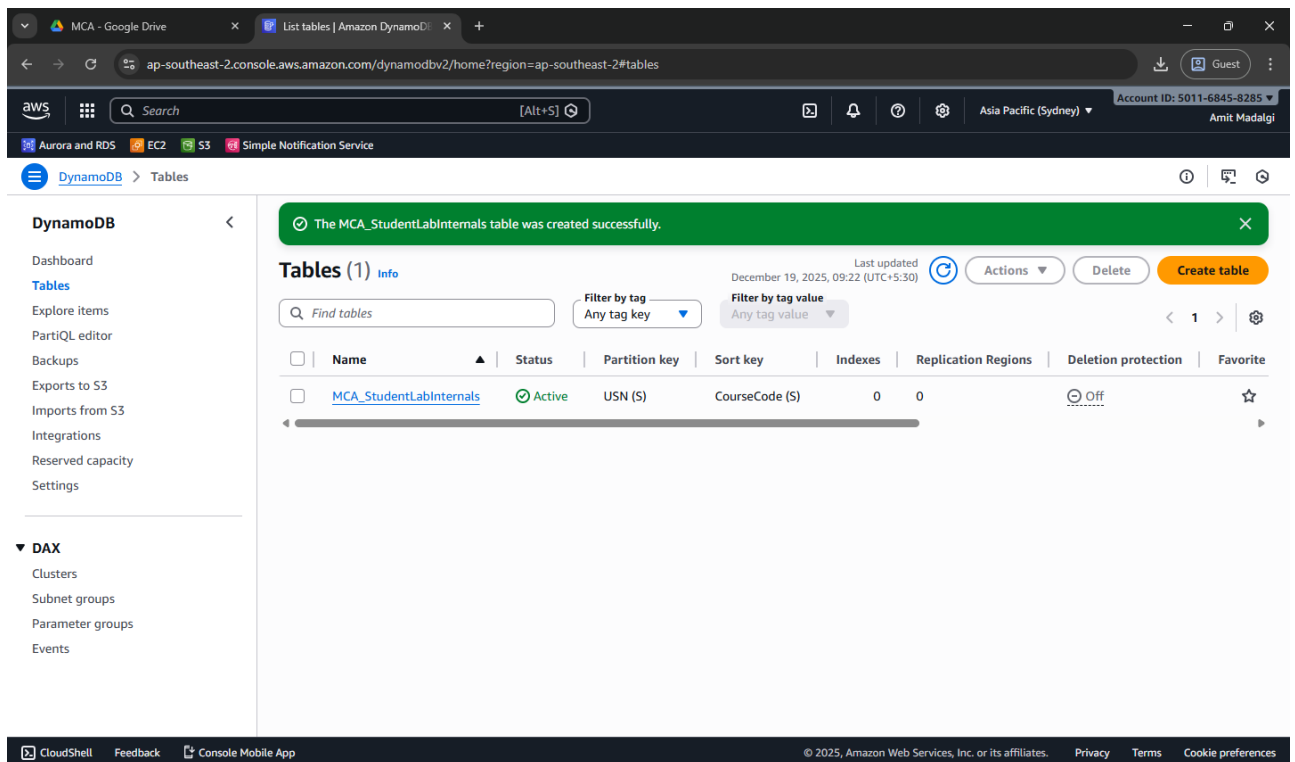
Step 3: Table settings

- Under Table settings, choose:
 - Default settings (recommended for lab)
- Ensure Capacity mode is: On-demand (default already selected)



Step 4: Create

- Click Create table
- Wait until table status becomes Active



Phase 2: Insert Items (Create Data)

Step 5: Open Table and Add Items

- Click the table: MCA_StudentLabInternals
- Click: Explore table items
- Click Create item

Step 6: Add Item 1 (Student 1 - Course 1)

- You will see attributes:
 - USN (String)
 - CourseCode (String)

Enter:

- USN = 1MS24MCA001
- CourseCode = CCL301

Now add additional attributes (click **Add new attribute**):

- Name (String) = Arun
- Semester (Number) = 3
- Attendance (Number) = 86
- IA1Marks (Number) = 18

Click **Create item**

Create item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attribute name	Value	Type	Remove
USN - Partition key	1MS24MCA001	String	
CourseCode - Sort key	CCL301	String	
Name	Arun	String	Remove
Semester	3	Number	Remove
Attendance	86	Number	Remove
IA1 Marks	18	Number	Remove

[Cancel](#) [Create item](#)

Step 7: Add Item 2 (Same student - another course)

Repeat Create item with:

- USN = 1MS24MCA001
- CourseCode = DBS301
- Name = Arun
- Semester = 3
- Attendance = 88
- IA1Marks = 20

Create item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Form | JSON view

Attributes Add new attribute

Attribute name	Value	Type	Remove
USN - Partition key	1MS24MCA001	String	
CourseCode - Sort key	DBS301	String	
Name	Arun	String	Remove
Semester	3	Number	Remove
Attendance	88	Number	Remove
IA1 Marks	20	Number	Remove

Cancel Create item

Step 8: Add Item 3 (Another student)

Create item:

- USN = 1MS24MCA002
- CourseCode = CCL301
- Name = Chitra
- Semester = 3
- Attendance = 74
- IA1Marks = 14

Create item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes

Attribute name	Value	Type	Remove
USN - Partition key	1MS24MCA002	String	
CourseCode - Sort key	CCL301	String	
Name	Chitra	String	Remove
Semester	3	Number	Remove
Attendance	74	Number	Remove
IA1 Marks	14	Number	Remove

[Add new attribute](#) [Cancel](#) [Create item](#)

You should now see 3 items in the table list.

DynamoDB

Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings

DAX
Clusters
Subnet groups
Parameter groups
Events

Select a table or index: Table - MCA_StudentLabInternals

Select attribute projection: All attributes

Filters - optional

[Run](#) [Reset](#)

Completed - Items returned: 0 - Items scanned: 0 - Efficiency: 100% - RCUs consumed: 2

Table: MCA_StudentLabInternals - Items returned (3)

Scan started on December 19, 2025, 09:26:10

	USN (String)	CourseCode (String)	Attendance	IA1 Marks	Name	Semester
☐	1MS24MCA002	CCL301	74	14	Chitra	3
☐	1MS24MCA001	DBS301	88	20	Arun	3
☐	1MS24MCA001	CCL301	86	18	Arun	3

Phase 3: Read Data (Get / Query / Scan)

Step 9: Get a single item (exact PK + SK)

- In “Explore table items”, use search/filter:

- Use USN = 1MS24MC001 and CourseCode = CCL301
- Open the item → verify all attributes

Key concept:

To uniquely identify an item in a PK+SK table, you must provide **both**.

The screenshot displays the AWS Management Console interface for Amazon DynamoDB. The left sidebar shows the navigation menu with options like Dashboard, Tables, Explore items, PartQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main content area shows the 'MCA_StudentLabInternals' table. The 'Filters - optional' section is active, showing two filters: 'CourseCode' equal to 'CCL301' and 'USN' equal to '1MS24MCA001'. The 'Run' button is highlighted. Below the filters, a green banner indicates 'Completed - Items returned: 1 - Items scanned: 3 - Efficiency: 33.33% - RCUs consumed: 2'. The table view shows one item with the following attributes: USN (String), CourseCode (String), Attendance, IA1 Marks, Name, and Semester.

USN (String)	CourseCode (String)	Attendance	IA1 Marks	Name	Semester
1MS24MCA001	CCL301	86	18	Arun	3

Step 10: Query – fetch all courses for one student

- Click Run query
- Set Partition key condition: USN equals 1MS24MCA001
- Run

The screenshot shows the AWS Management Console interface for the DynamoDB Item Explorer. The left sidebar contains navigation links for DynamoDB, DAX, and other services. The main content area displays the 'MCA_StudentLabInternals' table. The query filter is set to 'CourseCode' with the value '1MS24MCA001'. The results show two items for Arun, one for CCL301 and one for DBS301.

USN (String)	CourseCode (String)	Attendance	IA1 Marks	Name	Semester
1MS24MCA001	CCL301	86	18	Arun	3
1MS24MCA001	DBS301	88	20	Arun	3

Expected output:

- Items for Arun in both CCL301 and DBS301

Key concept:

Query works only with Partition Key (and optional Sort Key conditions). It is efficient.

Step 11: Scan (Not for large tables)

- Click Scan
- Run scan without filters

Expected output:

- All items (Arun + Chitra)

Key concept:

Scan reads the entire table → slow/costly for big tables.

The screenshot shows the AWS Management Console interface for Amazon DynamoDB. The left sidebar contains navigation links for DynamoDB, DAX, Clusters, Subnet groups, Parameter groups, and Events. The main content area is titled 'Explore items' and shows the table 'MCA_StudentLabInternals'. The 'Select a table or index' dropdown is set to 'Table - MCA_StudentLabInternals', and the 'Select attribute projection' dropdown is set to 'All attributes'. Below these, there are 'Filters - optional' and 'Run' and 'Reset' buttons. A green status bar indicates 'Completed - Items returned: 3 - Items scanned: 3 - Efficiency: 100% - RCUs consumed: 2'. The table 'MCA_StudentLabInternals - Items returned (3)' shows the following data:

	USN (String)	CourseCode (String)	Attendance	IA1 Marks	Name	Semester
☐	1MS24MCA002	CCL301	74	14	Chitra	3
☐	1MS24MCA001	CCL301	86	18	Arun	3
☐	1MS24MCA001	DBS301	88	20	Arun	3

Phase 4: Update Data

Step 12: Update IA1 marks of Chitra for CCL301

- Open item where:
USN = 1MS24MCA002
CourseCode = CCL301
- Click Edit
- Change: IA1Marks from 14 to 16
- Click Save changes

Verify updated value appears.

Edit item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Form | JSON view

Attributes Add new attribute

Attribute name	Value	Type	Remove
USN - Partition key	1MS24MCA002	String	
CourseCode - Sort key	CCL301	String	
Attendance	74	Number	Remove
IA1 Marks	16	Number	Remove
Name	Chitra	String	Remove
Semester	3	Number	Remove

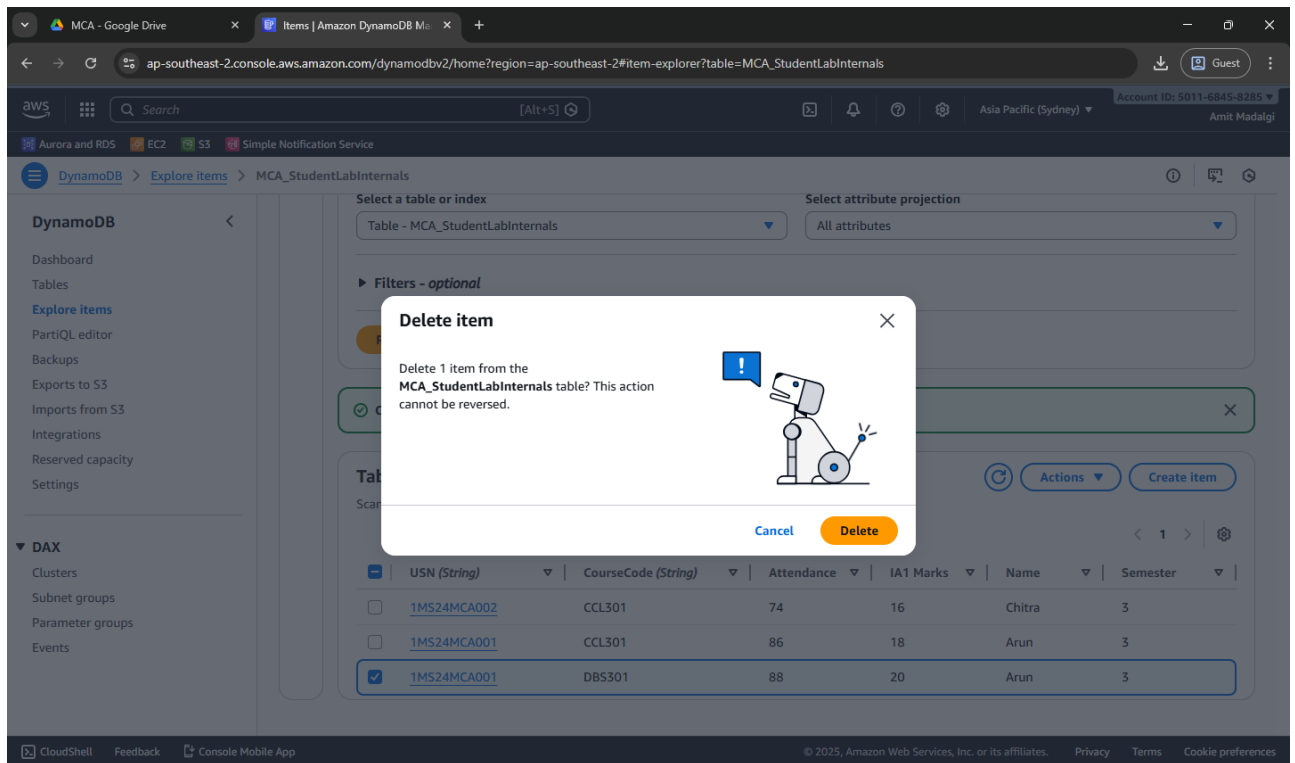
Cancel Save Save and close

Phase 5: Delete Data

Step 13: Delete one item (Arun's DBS301 record)

- Select item:
 - USN = 1MS24MCA001
 - CourseCode = DBS301
- Click **Delete**
- Confirm delete

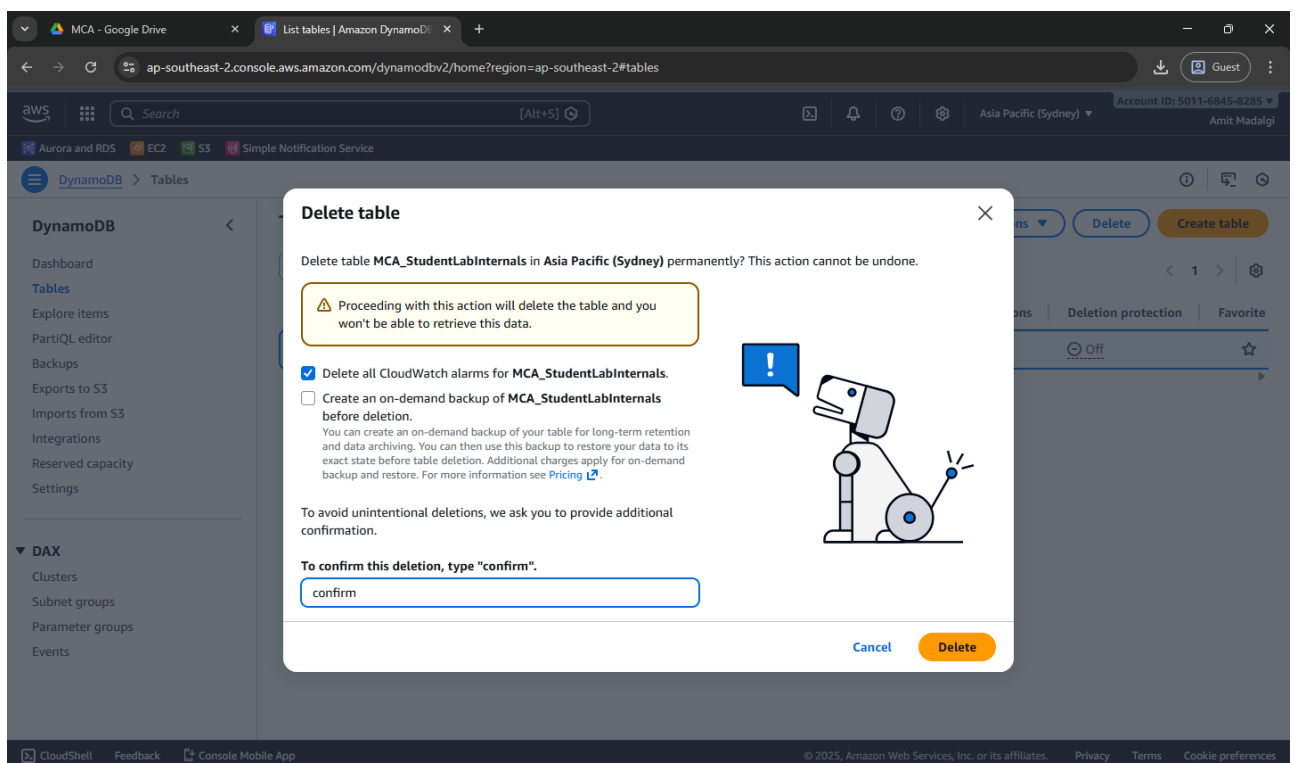
Now total items should reduce by 1.



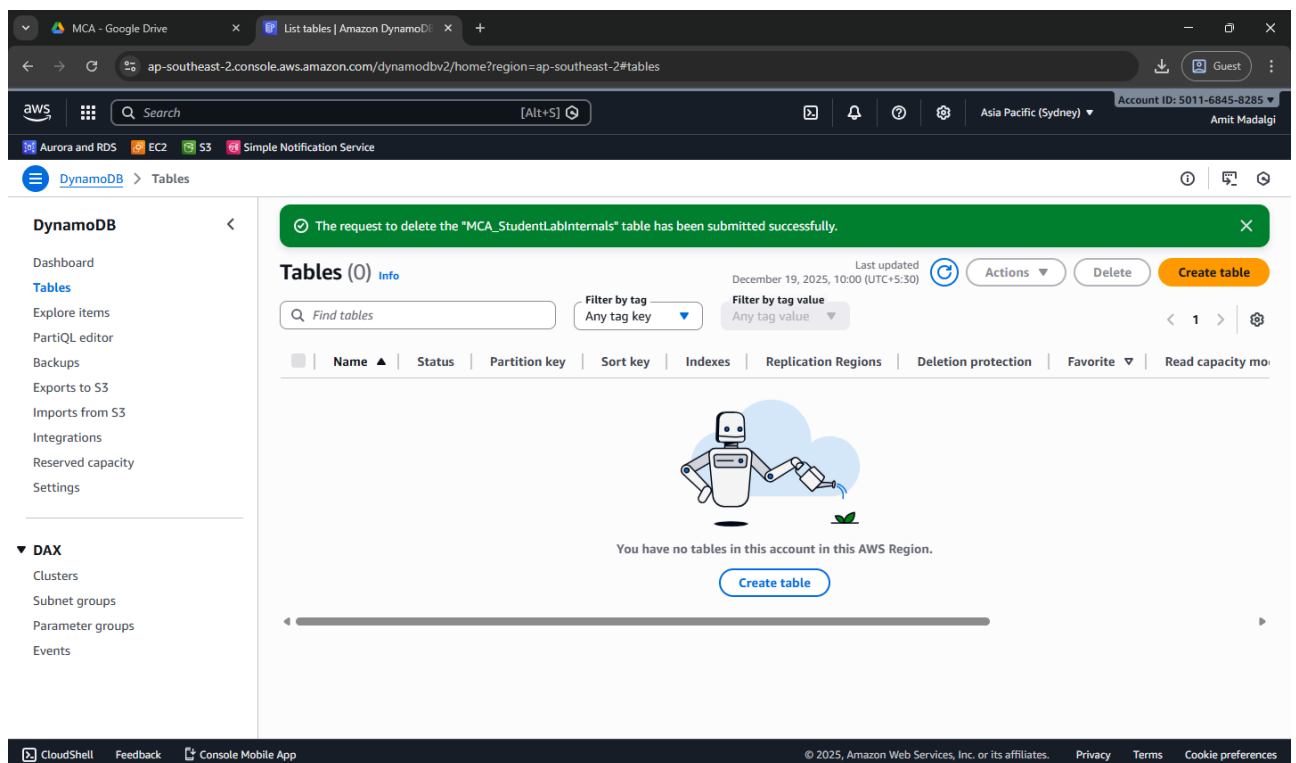
Phase 6: Cleanup (Must Do to avoid charges)

Step 15: Delete the Table

- DynamoDB → Tables
- Select MCA_StudentLabInternals
- Click Delete
- Type confirmation (if asked) and delete



Ensure table disappears from list.



DATE: 03-12-25

Exercise - 19: ElastiCache (Redis) as an In-Memory Cache

To implement Amazon ElastiCache (Redis) as an in-memory caching service by deploying a Redis cluster and performing basic cache operations from an EC2 instance.

Application-Level Data Flow (Logical)

Step 1: User requests data

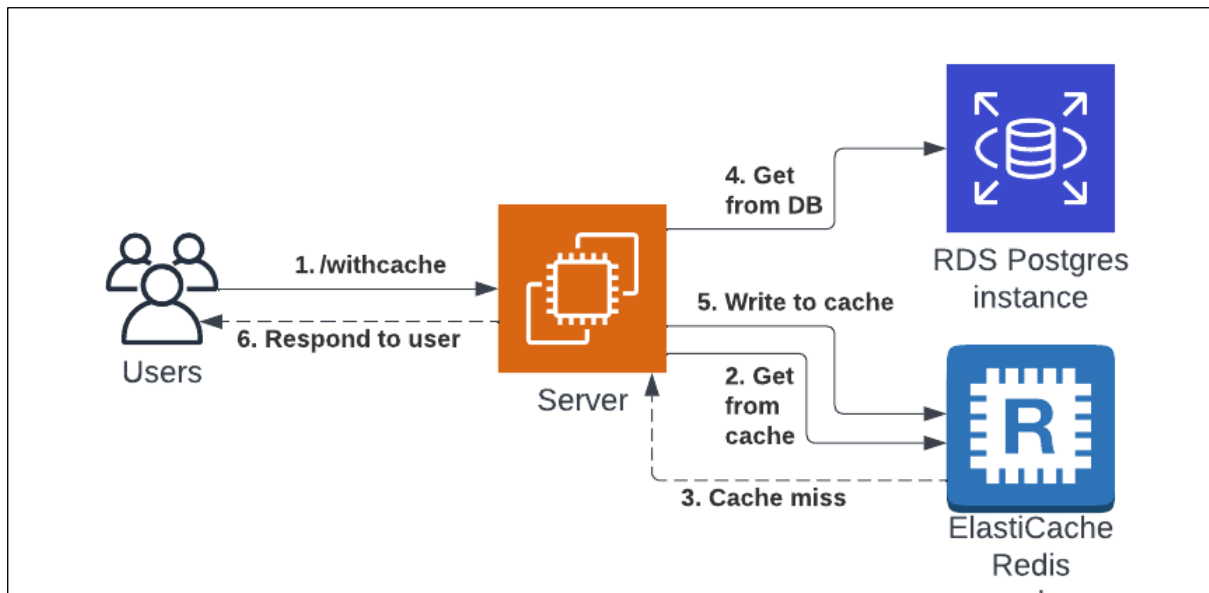
Step 2: Application checks ElastiCache (Redis)

Step 3: If data exists → return from cache (FAST)

Step 4: If data does not exist → fetch from RDS

Step 5: Store fetched data in ElastiCache

Step 6: Return data to user



This exercise provides **exposure** to an industry-used in-memory data store. Redis improves performance by serving frequently accessed data from memory and is typically used **in front of databases** in real-world applications.

Feature	RDS	ElastiCache
Storage	Disk	RAM
Data lifetime	Permanent	Temporary
Access speed	Slower	Very fast
TTL support	No	Yes

Phase-1 → EC2 + valkey-cli ready

Phase-2 → Redis cache created

Phase-3 → Connect EC2 → Redis and run commands

Phase-1: Launch EC2 Client using Amazon Linux 2023 (for Valkey / Redis Client)

Purpose of Phase-1

To create an EC2 instance using **Amazon Linux 2023** that will act as a **client machine** to connect to Amazon ElastiCache (Redis OSS) using **valkey-cli**.

Step 1: Select AWS Region

- Choose ONE region and stick to it for the entire exercise
ap-south-1 (Mumbai) (*recommended*)
- Ensure ElastiCache and EC2 will be in the SAME region

Step 2: Launch EC2 Instance

1. Go to **AWS Console** → **EC2**
2. Click **Launch instance**

Step 3: Configure Instance Basics

- Name: RedisClient-AL2023
- AMI: Select Amazon Linux 2023 AMI
- Instance Type: Select t3.micro (Free Tier eligible)

EC2

Instances

Launch an instance

Name and tags

Info

Name

RedisClient-AL2023

Add additional tags

▼ Application and OS Images (Amazon Machine Image)

Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Search

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Free tier eligible

Amazon Linux 2023 kernel-6.1 AMI

ami-0b3c832b6b7289e44 (64-bit (x86), uefi-preferred) / ami-083dd800ab194788b (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

▼ Summary

Number of instances

Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.9.2...[read more](#)

ami-0b3c832b6b7289e44

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier:

In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or

Cancel

Launch instance

Activate Windows

Go to Settings to activate Windows.

- Key Pair: Select an existing key pair OR create a new one (RSA, .pem)
- Network & Security
 - Network
 - VPC: Default VPC
 - Subnet: No preference
 - Auto-assign public IP: Enabled

EC2

Instances

Launch an instance

key1

Create new key pair

▼ Network settings

VPC - required | Info

vpc-0c0a06204e6990873 (default) 172.31.0.0/16

Subnet | Info

subnet-01ac6348fa52891c1 VPC: vpc-0c0a06204e6990873 Owner: 501168458285 Availability Zone: ap-southeast-2c (lapse2-az2) Zone type: Availability Zone IP addresses available: 4091 CIDR: 172.31.0.0/20

Auto-assign public IP | Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) | Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

Security group name - required

SG-RedisClient

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-./!@#%^&*~

▼ Summary

Number of instances | Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.9.2...read more
ami-0b3c832b6b7289e44

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or

Cancel

Launch instance

Preview code

- Security Group (VERY IMPORTANT)
Create a new security group:
 - Security Group Name: SG-RedisClient
 - Description: Security group for Redis client EC2

Inbound Rules

Type	Port	Source
SSH	22	My IP (or Anywhere for lab)

Do NOT add Redis (6379) here (The client does not listen on 6379)
Outbound rules: Allow all (default)

The screenshot shows the AWS Management Console interface for launching an EC2 instance. The 'Firewall (security groups)' section is expanded, showing the configuration for a new security group named 'SG-RedisClient'. The description is 'Security group for Redis client EC2'. An inbound rule is added for SSH on port 22 from 'My IP' (49.206.243.162/32). The 'Summary' section on the right shows 1 instance, Amazon Linux 2023 AMI, t3.micro instance type, and 8 GiB storage. A 'Launch instance' button is visible at the bottom right.

Step 5: Storage

- Keep default

Step 6: Launch Instance

- Click Launch instance
- Wait until Instance State = Running

Step 7: Connect to EC2

1. EC2 → Instances → select RedisClient-AL2023
2. Click Connect
3. Choose EC2 Instance Connect
4. Click Connect

You are now inside the EC2 terminal.

Step 8: Install Valkey Client

Run the following commands:

```
sudo dnf update -y
```

```
sudo dnf install -y valkey
```

Verify installation:

```
valkey-cli --version
```

Expected Output (example)

```
valkey-cli 8.x.x
```

This confirms the EC2 is ready as a Redis-compatible client.

Note: why install valkey and not redis??

Amazon Linux 2023 does not provide redis-cli directly.

AWS now provides **Valkey**, which is fully Redis-protocol compatible.

Hence, we use **valkey-cli** to connect to ElastiCache Redis.

Phase-2: Create Amazon ElastiCache (Redis OSS) Cluster

Purpose of Phase-2

To create an Amazon ElastiCache Redis OSS cluster that will act as an in-memory cache, accessible from the EC2 client created in Phase-1.

Step 1: Confirm AWS Region

- Ensure you are in the **same region** used in Phase-1
EC2 and ElastiCache must be in the SAME region and VPC.

Step 2: Open ElastiCache Console

1. AWS Console → **Services**
2. Select ElastiCache
3. Click Create cache

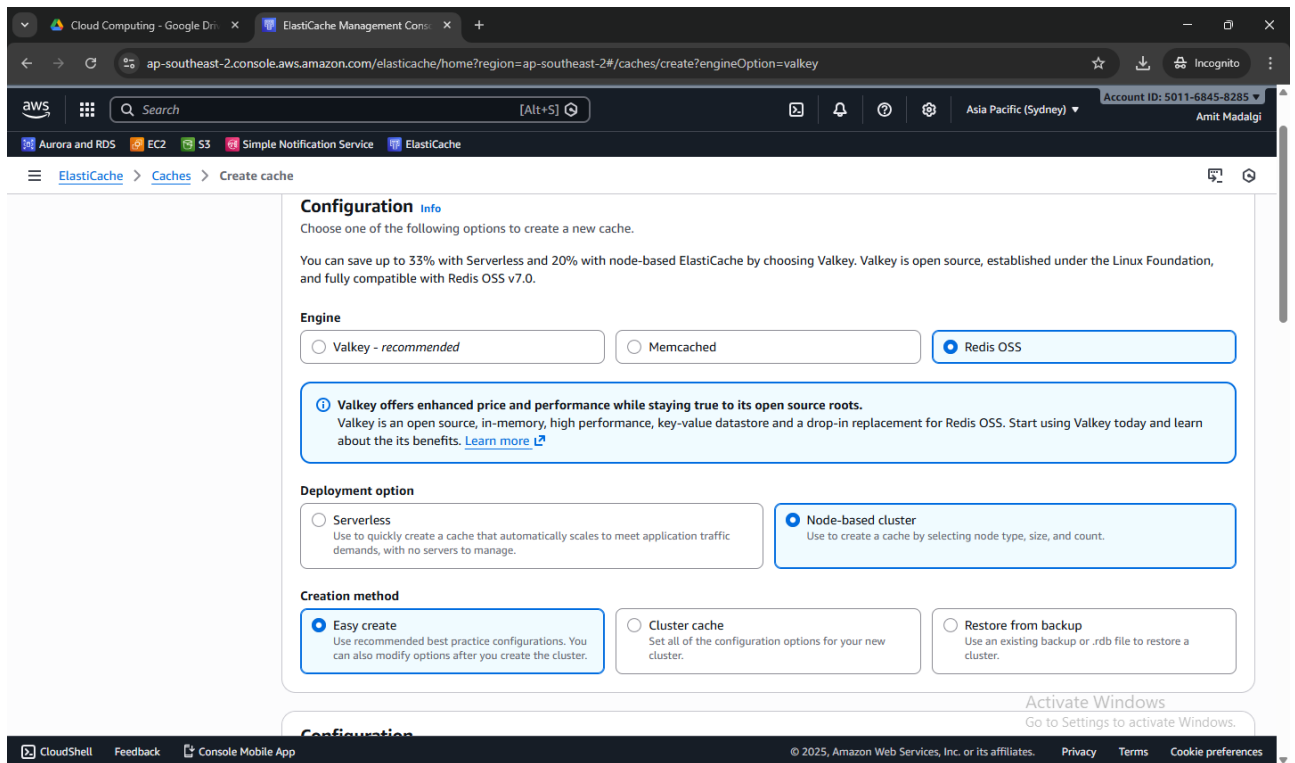
Step 3: Select Cache Engine

- **Engine: Redis OSS**

Step 4: Choose Deployment Settings

- Deployment option: Node-based cluster
- Creation method: Easy create

Easy create is used to avoid advanced production settings.



Step 5: Select Configuration

- **Configuration: Demo**

This automatically selects:

- A small, low-cost node (e.g., cache.t4g.micro)
- Suitable for labs and practice

Step 6: Provide Cluster Information

- Cache name: lab-redis
- Description: Optional (lab-redis)

Step 7: Configure Network and Subnet Group

Network

- Network type: IPv4

Subnet Group

- Select: Create a new subnet group
 - Subnet group name: redis-subnet-group
 - VPC: Default VPC
 - Subnets: Leave AWS auto-selected subnets unchanged

No manual subnet selection required.

Connectivity
Choose the IP version(s) this cluster will support. Then select an existing subnet group or create a new one.

Network type
Choose between IPv4, dual stack and IPv6
IPv4
Your resources will communicate only over the IPv4 protocol.

Subnet groups
☐ Choose existing subnet group ☒ Create a new subnet group

Name
redis-subnet-group
The name can have up to 255 characters, and must not contain spaces.

Description - optional
Description

VPC ID
The identifier for the VPC environment where your cluster is to run.
vpc-0c0a06204e6990873 [Create VPC](#)

Step 8: Configure Security

Security Group

- Create or select a security group:
 - Name: SG-RedisCache

Inbound Rule for Redis

Add one inbound rule to SG-RedisCache:

Type	Port	Source
Custom TCP	6379	SG-RedisClient

This allows only the EC2 client to access Redis.

Step 9: Authentication

- Authentication: Disabled

Step 10: Create Cache

1. Review all settings
2. Click Create
3. Wait until Status = Available

This may take a few minutes.

Amazon ElastiCache

Dashboard

Resources

- Valkey caches
- Memcached caches
- Redis OSS caches**
- Global datastores
- Reserved nodes
- Backups

Configurations

Redis OSS caches (1) [Info](#)

[Find Redis caches](#)

[Upgrade to Valkey](#) [View details](#) [Actions](#) [Create cache](#)

Cache name	Status	Description	Engine version	Configuration	Actions
lab-redis	Available	Easy created demo cluster on 2026-01...	7.1.0	cache.t4g.micro	Ja

Amazon ElastiCache announces Valkey starting as low as \$6/month
Introducing Valkey, an open source project that is a drop in replacement of Redis OSS, now available on ElastiCache, with up to 33% lower price than other engines. [Learn more](#)

[Create Valkey cache](#)

Step 11: Note the Redis Endpoint

1. Click on the cache name lab-redis
2. Copy the **Primary Endpoint**
 - Example: lab-redis.xxxxxx.cache.amazonaws.com

This endpoint will be used in **Phase-3** to connect from EC2.

Note:

We have created an in-memory Redis cache using Amazon ElastiCache. It runs inside the AWS VPC and does not have a public IP. Only our EC2 client is allowed to access it using port 6379.

Phase-3: Connect EC2 to ElastiCache Redis and Execute Cache Commands

Purpose of Phase-3

To connect the EC2 client (Amazon Linux 2023) to the ElastiCache Redis OSS cluster using valkey-cli and demonstrate basic in-memory cache operations.

Pre-checks - Before connecting, confirm:

- EC2 and ElastiCache are in the same AWS region
- EC2 security group = SG-RedisClient
- Redis security group = SG-RedisCache
- Redis security group allows port 6379 from SG-RedisClient
- Redis Status = Available
- You have copied the Primary Endpoint

Step 1: Connect to EC2

1. AWS Console → **EC2**
2. Select instance RedisClient-AL2023
3. Click **Connect**
4. Choose **EC2 Instance Connect**
5. Click **Connect**

You are now inside the EC2 terminal.

Step 2: Verify Valkey Client

Run: valkey-cli --version

Expected Output (example)

```
valkey-cli 8.x.x
```

This confirms the EC2 is ready to act as a Redis-compatible client.

Step 3: Connect to ElastiCache Redis

Use the Primary Endpoint from Phase-2.

```
valkey-cli -h <PRIMARY_ENDPOINT> -p 6379
```

Example: `valkey-cli -h lab-redis.xxxxxx.cache.amazonaws.com -p 6379`

Expected Result

You should see a prompt like:

```
lab-redis.xxxxxx.cache.amazonaws.com:6379>
```

This means the connection is successful.

Step 4: Execute Redis / Valkey Commands

These commands simulate what an application does internally.

Test Connectivity

PING

Expected Output

PONG

```
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> ping
PONG
```

Store and Retrieve Data (SET / GET)

SET course "Cloud Computing"

GET course

Expected Output

"Cloud Computing"

Demonstrates **key-value storage in memory**.

```
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> SET course "Cloud Computing"
OK
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> Get course
"Cloud Computing"
```

Counter Example (INCR)

INCR visits

INCR visits

GET visits

Expected Output

"2"

NOTE: Common real-world use - page views, hit counters.

```
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> INCR visits
(integer) 1
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> INCR visits
(integer) 2
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> GET visits
"2"
```

4.4 Set Data with Expiry (TTL)

SET notice "Results Published" EX 60

TTL notice

GET notice

Expected Output

- TTL shows a value ≤ 60
- GET returns:

"Results Published"

Shows **temporary cache data**.

```
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> SET notice "Results Published" EX 60
OK
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> TTL notice
(integer) 54
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> GET notice
"Results Published"
```

4.5 Verify Automatic Expiry

Wait ~60 seconds, then run:

GET notice

Expected Output

(nil)

Confirms data is **automatically removed from memory**.

```
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> GET notice
(nil)
```

4.6 Delete Data Manually

DEL course

GET course

Expected Output

(nil)

Step 5: Exit Redis Client

EXIT

```
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> DEL course
(integer) 1
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> GET course
(nil)
clustercfg.lab-redis.0e01xv.aps1.cache.amazonaws.com:6379> EXIT
[ec2-user@ip-172-31-14-150 ~]$
```

Note:

“The application first checks Redis.

If data is present, it is returned immediately from memory.

If not present, the application fetches data from the database and stores it in Redis with a TTL.

Redis automatically removes the data after expiry.”

Phase-3 Outcome

- EC2 successfully connected to ElastiCache Redis
- In-memory key–value operations verified
- Temporary storage and TTL behavior observed
- Redis used as a **cache**, not as a primary database

Common Errors & Quick Fix

Problem	Reason	Fix
Connection timeout	Wrong SG or region	Check SG-RedisCache inbound rule
Command hangs	Redis not Available	Wait & retry
(nil) output	Key expired	Expected behavior

Clean-Up (Mandatory)

Step 1: Delete Redis Cluster

- ElastiCache → Select lab-redis → Delete
- Disable snapshots

Step 2: Terminate EC2 Instance

- EC2 → Instances → Terminate RedisClient-EC2

Step 3: Optional

- Delete unused security groups

Redis Commands Explanation

- **SET** course "Cloud Computing"
- **GET** course
- **EXPIRE** course 30
- **TTL** course

The above commands simulate application caching behavior.

The SET command represents storing frequently accessed data in cache.

The EXPIRE command shows that cached data is temporary and stored in memory.

After expiry, the application would fetch fresh data from the database again.

Exercise - 22: CloudFormation – Launch EC2 (Amazon Linux 2023) + Install Apache using UserData

Create an EC2 instance using **AWS CloudFormation (YAML template)** and automatically install **Apache web server (httpd)** using **UserData**, then verify the website from a browser.

Pre-requisites

- Region set to **Asia Pacific (Mumbai) – ap-south-1**
- Basic knowledge of EC2 and Security Groups

Part A — Create Key Pair (One-time setup)

Step A1: Open EC2 Key Pairs

- AWS Console → Search **EC2**
- Left menu → **Key Pairs** (under “Network & Security”)
- Click **Create key pair**

Step A2: Create key pair

1. Name: pemkeypair (any name is fine)
2. Key pair type: **RSA**
3. Private key file format:
 1. **.pem** (recommended)

Click **Create key pair**

A file will download like: pemkeypair.pem

Note: CloudFormation uses **key pair NAME** (pemkeypair), not the file name.

Key pair

A key pair, consisting of a private key and a public key, is a set of security credentials that you use to prove your identity when connecting to an instance.

Name

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type [Info](#)

☒ RSA ☐ ED25519

Private key file format

☒ .pem
For use with OpenSSH
☐ .ppk
For use with PuTTY

Tags - optional
No tags associated with the resource.

[Add new tag](#)
You can add up to 50 more tags.

[Cancel](#) [Create key pair](#)

Part B — Create CloudFormation Template File (Amazon Linux 2023 + Apache)

Step B1: Create YAML file on your computer

- Open **Notepad**
- Paste the full YAML template given below
- Save as: **ec2-apache-al2023.yaml**
 - Save type: **All files**
 - Encoding: UTF-8 (if asked)

Full CloudFormation Template (Amazon Linux 2023)

Important:

- Do **not** add .pem anywhere.
- You will select Key Pair from dropdown during stack creation.

AWSTemplateFormatVersion: "2010-09-09"

Description: Simple EC2 with Apache using UserData

Parameters:

KeyName:

Type: AWS::EC2::KeyPair::KeyName

Resources:

WebSG:

Type: AWS::EC2::SecurityGroup

Properties:

GroupDescription: Allow SSH and HTTP

SecurityGroupIngress:

- { IpProtocol: tcp, FromPort: 22, ToPort: 22, CidrIp: 0.0.0.0/0 }
- { IpProtocol: tcp, FromPort: 80, ToPort: 80, CidrIp: 0.0.0.0/0 }

WebInstance:

Type: AWS::EC2::Instance

Properties:

InstanceType: t3.micro

KeyName: !Ref KeyName

SecurityGroups: [!Ref WebSG]

ImageId: !Sub "\${resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64}"

UserData:

Fn::Base64: |

#!/bin/bash

dnf -y update

dnf -y install httpd

systemctl enable --now httpd

echo "<h1>Hello from CloudFormation (AL2023)</h1>" > /var/www/html/index.html

Outputs:

WebsiteURL:

Value: !Sub "http://\${WebInstance.PublicDnsName}"

Part C — Create CloudFormation Stack (Console Steps)

Step C1: Open CloudFormation

- AWS Console → Search **CloudFormation**
- Click **Stacks**
- Click **Create stack** → **With new resources (standard)**

Step C2: Prepare template (select correct options)

1. Under **Prepare template**:
Select **Choose an existing template**
2. Under **Template source**:
Select **Upload a template file**
3. Click **Choose file** → select ec2-apache-al2023.yaml
4. Click **Next**

Prerequisite - Prepare template

You can also create a template by scanning your existing resources in the [laC generator](#).

Prepare template

Every stack is based on a template. A template is a JSON or YAML file that contains configuration information about the AWS resources you want to include in the stack.

☒ Choose an existing template

Upload or choose an existing template.

☐ Build from Infrastructure Composer

Create a template using a visual builder.

Specify template [Info](#)

This [GitHub repository](#) contains sample CloudFormation templates that can help you get started on new infrastructure projects. [Learn more](#)

Template source

Selecting a template generates an Amazon S3 URL where it will be stored. A template is a JSON or YAML file that describes your stack's resources and properties.

☐ Amazon S3 URL

Provide an Amazon S3 URL to your template.

☒ Upload a template file

Upload your template directly to the console.

☐ Sync from Git

Sync a template from your Git repository.

Upload a template file

[Choose file](#)

ec2-apache-al2023.yaml

JSON or YAML formatted file

S3 URL: <https://s3.ap-south-1.amazonaws.com/cf-templates-wrpjcdq5uc4d-ap-south-1/2026-01-02T041318.889Zoas-ec2-apache-al2023.yaml>

[View in Infrastructure Composer](#)

Part D — Specify Stack Details

Step D1: Stack name

- Stack name: EC2-Apache-AL2023

Click **Next**

Step D2: Parameters

- Under **KeyName**, select your key pair name from dropdown
Example: pemkeypair

Click **Next**

Specify stack details

Provide a stack name

Stack name

Stack name must contain only letters (a-z, A-Z), numbers (0-9), and hyphens (-) and start with a letter. Max 128 characters. Character count: 17/128.

Parameters

Parameters are defined in your template and allow you to input custom values when you create or update a stack.

KeyName
Select an existing EC2 Key Pair to enable SSH access

[Cancel](#)
[Previous](#)
[Next](#)

Part E — Configure Stack Options (keep default)

- Leave everything as default
- Click **Next**

Part F — Review and Create

- Scroll down
- Click **Create stack**

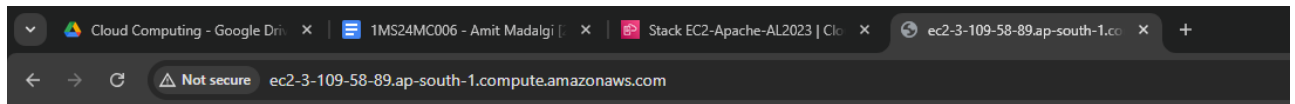
CloudFormation > Stacks				
Stacks (1)				
Search by stack name Delete Update stack Stack actions Create stack				
Filter status Active View nested				
Stack name	Status	Created time	Description	
EC2-Apache-AL2023	CREATE_IN_PROGRESS	2026-01-02 09:35:28 UTC+0530	Launch EC2 (Amazon Linux 2023) and install Apache (httpd) using UserData	

Part G — Monitor Stack Creation

- Wait for Stack status to become:
CREATE_COMPLETE
- Open the stack → click **Outputs** tab
- Copy **WebsiteURL**
- Paste URL in browser

Expected Output in browser:

Apache Installed via CloudFormation UserData (Amazon Linux 2023)!



Apache Installed via CloudFormation UserData (Amazon Linux 2023)!

Part H — Verify in EC2 (Optional)

- Go to **EC2 → Instances**
- Instance should be running
- Security group should allow:
- SSH 22
- HTTP 80

Instances (1/1) Info		Refresh	Connect	Instance state ▾	Actions ▾	Launch instances	▼
Find Instance by attribute or tag (case-sensitive)		All states ▾					
Instance state = running X		Clear filters		< 1 > ⚙️			
☒	Name 🔗 ▾	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status	Availability Zone ▾
☒		i-054a03639135f8233	Running 🔍 🔍	t3.micro	3/3 checks passed View alarms +		ap-south-1b

Troubleshooting (Common Errors)

1) Website not opening

Check:

- EC2 instance status checks are **2/2 passed**
- Security group has **port 80 open**
- Wait 2–3 minutes (UserData takes time)

2) Stack went to ROLLBACK

Go to stack → **Events tab** → check the failure reason

Most common reason:

- Wrong Key Pair selected / key pair does not exist

Part I — Clean-up

To avoid charges:

- CloudFormation → Stacks
- Select EC2-Apache-AL2023
- Click **Delete**
- Confirm

This deletes EC2 + Security Group automatically.

DATE: 17-12-25

Exercise - 23: EC2 + S3 (Static Content Pulled from S3 using CloudFormation)

Create an **S3 bucket** using CloudFormation.

Upload a static HTML file (index.html) to S3.

Launch **EC2 (Amazon Linux 2023) + Apache** using CloudFormation.

EC2 will **pull index.html from S3** and host it via Apache.

Pre-requisites

- Region: **Mumbai (ap-south-1)**
 - A Key Pair exists (example: pemkeypair)
-

PART 0 — One-time: Create Key Pair

EC2 → Key Pairs → Create key pair

- Name: pemkeypair
 - Format: .pem
Download it and keep safe.
-

PART 1 — Stack-1: Create S3 Bucket (CloudFormation)

Step 1.1: Create S3 template file

Create a file: **stack1-s3-bucket.yaml** and paste:

```
AWSTemplateFormatVersion: "2010-09-09"
```

```
Description: Create an S3 bucket to store website content (index.html)
```

```
Resources:
```

```
WebsiteBucket:
```

```
    Type: AWS::S3::Bucket
```

```
Outputs:
```

BucketName:

Description: S3 Bucket Name (use this to upload index.html)

Value: !Ref WebsiteBucket

Step 1.2: Create stack

CloudFormation → Stacks → Create stack → With new resources

- Choose an existing template
- Upload a template file → stack1-s3-bucket.yaml
- Stack name: S3-Website-Bucket
- Create stack

The screenshot shows the AWS CloudFormation console. On the left, the 'Stacks (1)' sidebar lists the 'S3-Website-Bucket' stack with a status of 'CREATE_COMPLETE'. The main panel displays the 'S3-Website-Bucket' stack details, with the 'Events' tab selected. The events table shows the following:

Operation ID	Timestamp	Logical ID	Status	Detailed status
9d450a89-8d6a-4ac6-b8d2-3eff840ad96c	2026-01-07 11:32:27 UTC+0530	S3-Website-Bucket	CREATE_COMPLETE	-
9d450a89-8d6a-4ac6-b8d2-3eff840ad96c	2026-01-07 11:32:26 UTC+0530	WebsiteBucket	CREATE_COMPLETE	-
9d450a89-8d6a-4ac6-b8d2-3eff840ad96c	2026-01-07 11:32:13	WebsiteBucket	CREATE_IN_PROGRESS	-

Step 1.3: Copy bucket name

Open stack → Outputs → copy **BucketName**

The screenshot shows the 'Outputs (1)' tab for the 'S3-Website-Bucket' stack. The output table is as follows:

Key	Value	Description	Export name
BucketName	s3-website-bucket-websitebucket-hbr6c0wjbbmc	S3 Bucket Name (use this to upload index.html)	-

PART 2 — Upload Static Content to S3 (Manual)

Step 2.1: Create **index.html** on your PC

Create a file named **index.html** with this content:

```
<!DOCTYPE html>

<html>

<head>

  <title>EC2 + S3 Demo</title>

</head>

<body>

  <h1>Hello! This page was pulled from S3 to EC2 automatically.</h1>

  <p>Deployed using CloudFormation + UserData</p>

</body>

</html>
```

Step 2.2: Upload to S3 bucket

S3 → Buckets → open your bucket (from Output) → Upload

- Upload **index.html**
- Keep it at **root** (no folder)
- Upload

Now S3 has: s3://<your-bucket-name>/index.html

Amazon S3

Upload succeeded

For more information, see the [Files and folders](#) table.

Upload: status

After you navigate away from this page, the following information is no longer available.

Summary

Destination s3://s3-website-bucket-websitebucket-hbr6c0wjbbmc	Succeeded 1 file, 221.0 B (100.00%)	Failed 0 files, 0 B (0%)
---	--	---

PART 3 — Stack-2: Launch EC2 + Apache + Pull from S3

This stack will:

- Create IAM Role (permission to read the bucket)
- Create Security Group (22, 80)
- Launch EC2 (Amazon Linux 2023)
- Install Apache
- Copy index.html from S3 to /var/www/html/index.html

Step 3.1: Create EC2 template file

Create a file: **stack2-ec2-pull-from-s3.yaml** and paste:

```
AWSTemplateFormatVersion: "2010-09-09"

Description: Launch EC2 (Amazon Linux 2023), install Apache, and pull index.html from S3

Parameters:

  KeyName:

    Type: AWS::EC2::KeyPair::KeyName

    Description: Select an existing EC2 Key Pair to enable SSH access

  BucketName:

    Type: String
```

Description: Enter the S3 bucket name created in Stack-1 (Output)

SubnetId:

Type: AWS::EC2::Subnet::Id

Description: Select a PUBLIC subnet (default VPC public subnet recommended)

VpcId:

Type: AWS::EC2::VPC::Id

Description: Select the VPC (default VPC recommended)

Resources:

WebServerRole:

Type: AWS::IAM::Role

Properties:

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

Service: ec2.amazonaws.com

Action: sts:AssumeRole

Policies:

- PolicyName: S3ReadOnlyForWebsiteBucket

PolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Action:

- s3:GetObject

- s3:ListBucket

Resource:

- !Sub "arn:aws:s3:::\${BucketName}"

- !Sub "arn:aws:s3:::\${BucketName}/*"

WebServerInstanceProfile:

Type: AWS::IAM::InstanceProfile

Properties:

Roles:

- !Ref WebServerRole

WebServerSG:

Type: AWS::EC2::SecurityGroup

Properties:

GroupDescription: Allow SSH (22) and HTTP (80)

VpcId: !Ref VpcId

SecurityGroupIngress:

- IpProtocol: tcp

FromPort: 22

ToPort: 22

CidrIp: 0.0.0.0/0

- IpProtocol: tcp

FromPort: 80

ToPort: 80

CidrIp: 0.0.0.0/0

WebServerInstance:

Type: AWS::EC2::Instance

Properties:

InstanceType: t2.micro

KeyName: !Ref KeyName

SubnetId: !Ref SubnetId

SecurityGroupIds:

- !Ref WebServerSG

IamInstanceProfile: !Ref WebServerInstanceProfile

Amazon Linux 2023 AMI for Mumbai (ap-south-1)

ImageId: ami-02b49a24cfb95941c

UserData:

Fn::Base64: !Sub |

#!/bin/bash

dnf update -y

dnf install -y httpd

systemctl enable httpd

systemctl start httpd

```
# pull index.html from S3 to Apache web root
```

```
aws s3 cp s3://${BucketName}/index.html /var/www/html/index.html
```

```
systemctl restart httpd
```

Outputs:

WebsiteURL:

Description: Open this URL in browser

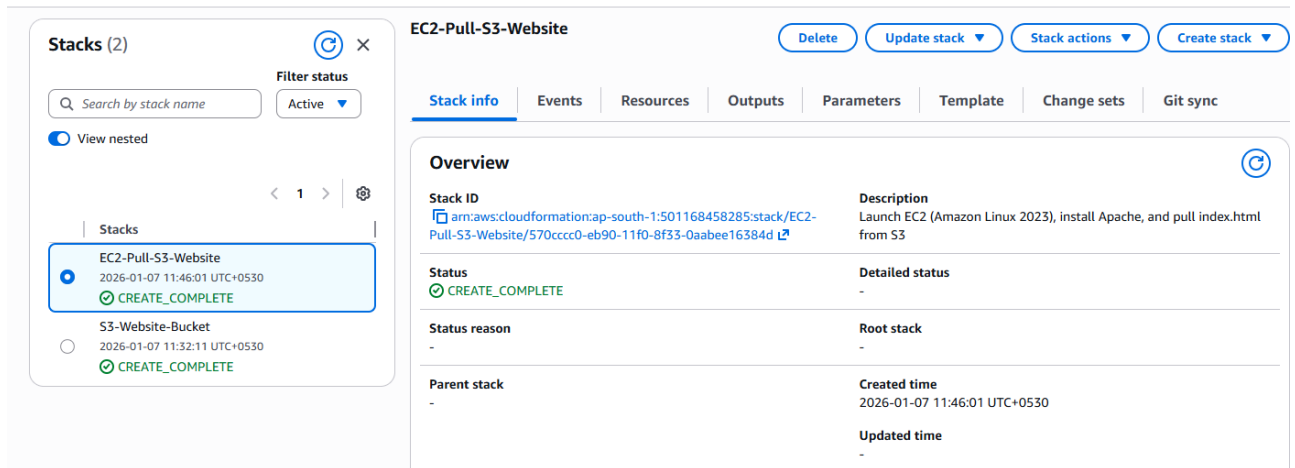
Value: !Sub "http://\${WebServerInstance.PublicDnsName}"

Step 3.2: Create stack

CloudFormation → Stacks → Create stack

- Upload template → stack2-ec2-pull-from-s3.yaml
- Stack name: EC2-Pull-S3-Website
- Parameters:
 - **KeyName**: select your key pair (e.g., pemkeypair)
 - **BucketName**: paste bucket name from Stack-1 Output
 - **VpcId**: select **default VPC**
 - **SubnetId**: select a **PUBLIC subnet** in default VPC
(usually the subnet name shows “public” or you can pick any default subnet that gives public IP; default subnets generally work)

Create stack → wait for **CREATE_COMPLETE**

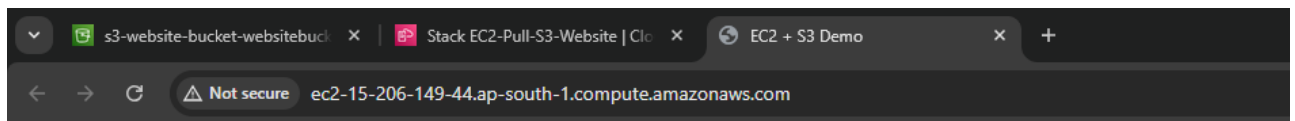


PART 4 — Verify Output

Stack-2 → **Outputs** → copy **WebsiteURL** → open in browser.

Expected page:

“Hello! This page was pulled from S3 to EC2 automatically...”



Hello! This page was pulled from S3 to EC2 automatically.

Deployed using CloudFormation + UserData

Troubleshooting (common)

1) Website not opening

- Wait 1–2 minutes (UserData takes time)
- Ensure Security Group has **HTTP 80 open**
- Ensure you selected a **public subnet** (needs internet to reach S3)

2) Stack-2 fails with S3 access error

- BucketName typed wrong
 - index.html not uploaded at root of bucket
-

Clean-up

Delete Stack-2 first

CloudFormation → select EC2-Pull-S3-Website → Delete

Empty the S3 bucket

S3 → bucket → delete index.html (empty bucket)

Delete Stack-1

CloudFormation → select S3-Website-Bucket → Delete

Viva-ready questions

1. What is the purpose of **UserData** in EC2?
2. Why do we need an **IAM Role + Instance Profile**?
3. Why is S3 bucket not made public in this lab?
4. What happens when a CloudFormation stack is deleted?

Which ports are required for web hosting and SSH?

Exercise - 24:

AWS Lambda: Input Processing, Business Logic Execution, and CloudWatch Logging

Create one Lambda function that:

1. prints a welcome message
2. reads JSON input event
3. performs business logic (grade calculation)
4. writes logs → view them in CloudWatch Logs

Yes — this is the **first combined Lambda lab exercise** (basics + input + logic + logging).
Below is a **complete step-by-step procedure** using **AWS Console**.

Create the Lambda Function

Step 1: Open Lambda

AWS Console → Search Lambda → Open AWS Lambda

Step 2: Create function

Click Create function

- Select - Author from scratch
- Function name: StudentGradeLogger
- Runtime: Python 3.11
- Architecture: x86_64 (default)

The screenshot shows the 'Create function' page in the AWS Lambda console. At the top, there's a 'Create function' header with an 'Info' link. Below it, a message says 'Choose one of the following options to create your function.' There are three radio button options: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. The 'Author from scratch' option has a sub-note: 'Start with a simple Hello World example.' Below these options is a 'Basic information' section. It contains a 'Function name' field with the value 'StudentGradeLogger' and a note: 'Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).' The 'Runtime' section shows 'Python 3.11' selected from a dropdown menu, with a note: 'Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.' There's a 'Durable execution - new' section with an 'Enable' checkbox and a note: 'Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Supports Python and Node.js runtimes. View pricing.' The 'Architecture' section shows 'x86_64' selected from a dropdown menu, with a note: 'Choose the instruction set architecture you want for your function code.'

Step 3: Permissions (Execution Role)

Under “Permissions”

- Choose: Create a new role with basic Lambda permissions

This automatically gives permission to write logs to CloudWatch.

Click **Create function**

✔ Successfully created the function `StudentGradeLogger`. You can now change its code and configuration. To invoke your function with a test event, choose "Test".

StudentGradeLogger

Throttle Copy ARN Actions

▼ Function overview Info

Diagram | Template

 **StudentGradeLogger**

 Layers (0)

+ Add trigger

+ Add destination

Export to Infrastructure Composer Download

Description
-

Last modified
1 second ago

Function ARN
 `arn:aws:lambda:ap-south-1:501168458285:function:StudentGradeLogger`

Function URL | Info
-

Add Code (Business Logic + Logs)

Step 4: Paste this code

In **Code** tab → `lambda_function.py` → paste and **Deploy**

```
import json

def lambda_handler(event, context):

    # Welcome message

    print("Lambda invoked successfully")


    # Read input from event

    student_name = event["StudentName"]

    marks = event["Marks"]


    # Business logic: grade calculation

    if marks >= 75:

        result = "Pass"

        grade = "A"
```

```
else:

    result = "Fail"

    grade = "F"


# Log output

print("Student:", student_name)

print("Marks:", marks)

print("Grade:", grade)

print("Result:", result)


# Return response

return {

    "statusCode": 200,

    "body": json.dumps({

        "StudentName": student_name,

        "Marks": marks,

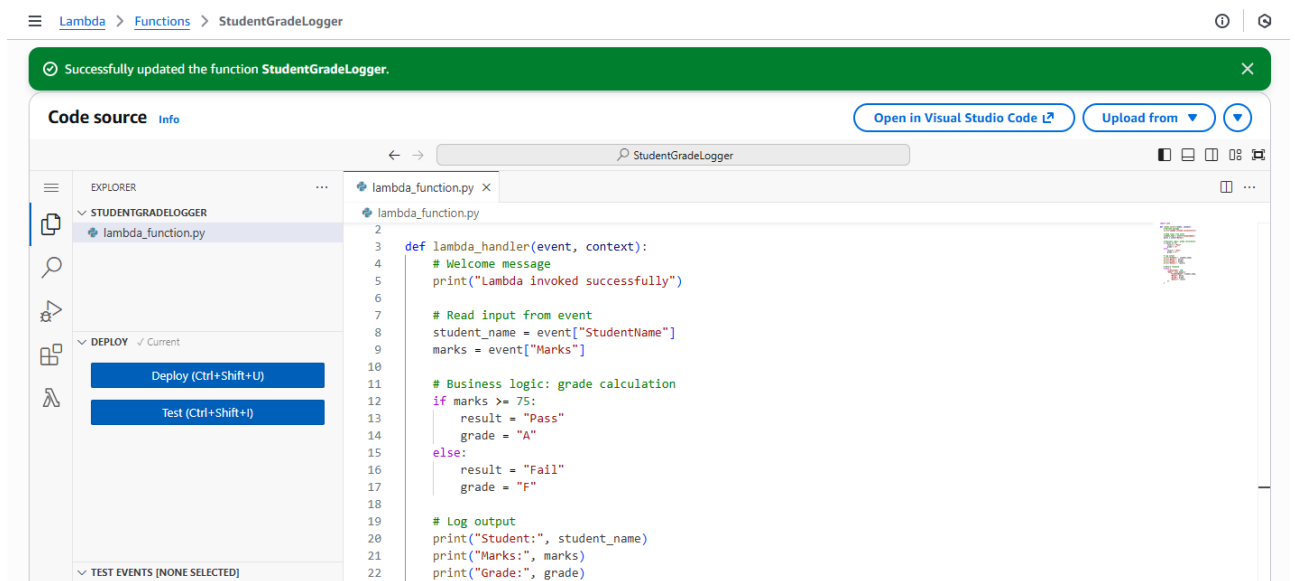
        "Grade": grade,

        "Result": result

    })

}
```

Click **Deploy**.



Test with Input Events (JSON)

Step 5: Create a test event (Valid case)

Go to Test (top right) → Configure test event

- Event name: ValidInput
- Paste this JSON:

```
{
  "StudentName": "Anita",
  "Marks": 86
}
```

Click **Save**

Invocation type

☒ Synchronous

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous

Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

ValidInput

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private

This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable

This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

Last fetched 1/6/2026, 11:34:07 AM

Event JSON

Format JSON

1 * {

2 "StudentName": "Anita",

3 "Marks": 86

4 }

5 |

Step 6: Run test

Click Test

Expected response (sample):

- statusCode: 200
- body will include Grade A and Result Pass

Executing function: succeeded ([logs](#))

Details

```
{
  "statusCode": 200,
  "body": "{\"StudentName\": \"Anita\", \"Marks\": 86, \"Grade\": \"A\", \"Result\": \"Pass\"}"
}
```

Summary

Code SHA-256 HAPq9EReJVEC5gLavtr/gyd5vZtd9eiUGF932t0jBxY=	Execution time 60 seconds ago
Function version \$LATEST	Request ID 6a574c18-6f47-4be0-995c-10dd8cec3ce3
Duration 1.66 ms	Billed duration 82 ms
Resources configured 128 MB	Max memory used 34 MB
Init duration 79.42 ms	
Log output The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.	

Step 7: Test invalid input (Missing Marks)

Create another test event:

- Event name: MissingMarks

```
{  
  
  "StudentName": "Rahul"  
  
}
```

Run Test

Expected:

- statusCode: 400
- message: Missing 'Marks'

 Executing function: succeeded ([logs](#))
▼ Details

Summary

Code SHA-256
HAPq9EReJVEC5gLavtc/gyd5vZtd9eiUGF932t0jBxY=

Execution time
0 seconds ago

Log output

[Click here](#) to view the corresponding CloudWatch log group.

Step 8: Test invalid marks range

Event name: InvalidMarks

```
{  
  
  "StudentName": "Priya",  
  
  "Marks": 150  
  
}
```

Expected:

- statusCode: 400
- message: Marks must be 0–100

 Executing function: succeeded ([logs](#))
▼ Details

Summary

Code SHA-256
HAPq9EReJVEC5gLavtc/gyd5vZtd9eiUGF932t0jBxY=

Execution time
0 seconds ago

Log output

[Click here](#) to view the corresponding CloudWatch log group.

View Logs in CloudWatch

Step 9: Open logs from Lambda directly

On the Lambda function page:

- Go to **Monitor** tab
- Click **View CloudWatch logs**

You will see:

- Log group: /aws/lambda/StudentGradeLogger
- Open latest **log stream**
- You should see all print() outputs:
 - “Lambda invoked successfully!”
 - “Received event...”
 - grade/result logs
 - error logs for invalid tests

No older events at this moment. Retry	
▶ 2026-01-06T06:15:05.330Z	INIT_START Runtime Version: python:3.11.v109 Runtime Version ARN: arn:aws:lambda:ap-south-1::runtime:49f733259c7ce7e0...
▶ 2026-01-06T06:15:05.413Z	START RequestId: 6a574c18-6f47-4be0-995c-10dd8cec3ce3 Version: \$LATEST
▶ 2026-01-06T06:15:05.414Z	Lambda invoked successfully
▶ 2026-01-06T06:15:05.414Z	Student: Anita
▶ 2026-01-06T06:15:05.414Z	Marks: 86
▶ 2026-01-06T06:15:05.414Z	Grade: A
▶ 2026-01-06T06:15:05.414Z	Result: Pass
▶ 2026-01-06T06:15:05.416Z	END RequestId: 6a574c18-6f47-4be0-995c-10dd8cec3ce3
▶ 2026-01-06T06:15:05.416Z	REPORT RequestId: 6a574c18-6f47-4be0-995c-10dd8cec3ce3 Duration: 1.66 ms Billed Duration: 82 ms Memory Size: 128 MB M...
▶ 2026-01-06T06:19:58.697Z	START RequestId: 8fcdf5ae-ec04-479a-80dd-9c191ac9f568 Version: \$LATEST
▶ 2026-01-06T06:19:58.716Z	Lambda invoked successfully
▶ 2026-01-06T06:19:58.717Z	[ERROR] KeyError: 'Marks' Traceback (most recent call last): File "/var/task/lambda_function.py", line 9, in lambda...
▶ 2026-01-06T06:19:58.738Z	END RequestId: 8fcdf5ae-ec04-479a-80dd-9c191ac9f568

Cleanup (Avoid charges, keep account clean)

Lambda itself usually costs nothing in small use, but cleanup is good practice:

1. Lambda → Functions → select StudentGradeLogger → **Delete**
2. CloudWatch → Log groups → find /aws/lambda/StudentGradeLogger → **Delete**
3. IAM role created (optional):
 - IAM → Roles → search role name created for Lambda → delete (only if not used elsewhere)

DATE: 24-12-25

Exercise - 25:

Mini Project: Event-Driven Notification System using AWS Lambda and Amazon SNS

Simulating real-time alerts from events using serverless computing

- An event in JSON format is given as input to an AWS Lambda function.
- The Lambda function processes the event and generates a notification message.
- The message is published to an Amazon SNS topic.
- SNS sends the notification to the subscribed email address.
- The execution details are verified using CloudWatch Logs.

Create SNS Topic + Email Subscription

Step 1: Open SNS

AWS Console → Search SNS → Open Simple Notification Service

Step 2: Create a Topic

SNS → Topics → Create topic

- Type: Standard
 - Name: NotificationTopic
- Click Create topic

The screenshot shows the AWS SNS console interface for a topic named "NotificationTopic". At the top, there are buttons for "Edit", "Delete", and "Publish message". Below this is a "Details" section with a table containing the following information:

Name	ARN	Display name	Type
NotificationTopic	arn:aws:sns:ap-south-1:501168458285:NotificationTopic	-	Standard

Below the table, it shows "Topic owner: 501168458285". At the bottom of the console, there is a "Subscriptions" section with a search bar and a table with columns: ID, Endpoint, Status, and Protocol. The table is currently empty, and a message "No subscriptions found" is displayed at the bottom.

Step 3: Copy Topic ARN

Open the created topic → copy Topic ARN

(You will paste it into Lambda code later)

Step 4: Create an Email Subscription

Inside the same topic:

Click Create subscription

- Protocol: Email
 - Endpoint: (your email id)
- Click Create subscription

Create subscription

Details

Topic ARN

Protocol
The type of endpoint to subscribe

Endpoint
An email address that can receive notifications from Amazon SNS.

After your subscription is created, you must confirm it. [Info](#)

Step 5: Confirm Subscription

Open your email inbox → find AWS SNS confirmation mail → click Confirm subscription
→ Status in SNS should become Confirmed.

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Subscription: 815328c4-95bf-4cbd-8b73-7f95f7f8d84b [Edit](#) [Delete](#)

Details

ARN
 arn:aws:sns:ap-south-1:501168458285:NotificationTopic:815328c4-95bf-4cbd-8b73-7f95f7f8d84b

Endpoint
amitrlsbca@gmail.com

Topic
[NotificationTopic](#)

Subscription Principal
arn:aws:iam::501168458285:root

Status
✔ Confirmed

Protocol
EMAIL

Create Lambda Function

Step 6: Open Lambda

AWS Console → Search Lambda → Open AWS Lambda

Step 7: Create function

Click Create function

- Select: Author from scratch
 - Function name: NotificationSimulator
 - Runtime: Python 3.11
 - Permissions: Create a new role with basic Lambda permissions
- Click Create function

Add Lambda Code (with SNS Publish)

Step 8: Paste code in lambda_function.py

Lambda → **Code** tab → open lambda_function.py → remove existing code → paste:

Replace <SNS_TOPIC_ARN> with your copied Topic ARN.

```
import json
import boto3

sns = boto3.client('sns')

TOPIC_ARN = "<SNS_TOPIC_ARN>"
```

```

def lambda_handler(event, context):
    print("Notification Simulator invoked")

    event_type = event["eventType"]
    user = event["user"]

    if event_type == "ORDER_PLACED":
        message = f"Hi {user}, your order is placed successfully."
        priority = "NORMAL"

    elif event_type == "PAYMENT_FAILED":
        message = f"Hi {user}, your payment has failed. Please retry."
        priority = "HIGH"

    elif event_type == "LOW_ATTENDANCE":
        message = f"Hi {user}, your attendance is low. Please take action."
        priority = "HIGH"

    else:
        message = f"Hi {user}, unknown event received."
        priority = "LOW"

    print("Event Type:", event_type)
    print("Message:", message)
    print("Priority:", priority)

    # Publish to SNS (Actual notification delivery)
    sns.publish(TopicArn=TOPIC_ARN, Message=message, Subject=f"{event_type} [{priority}]")

    return {
        "statusCode": 200,
        "body": json.dumps({
            "EventType": event_type,
            "Message": message,
            "Priority": priority
        })
    }

```

Click **Deploy**

Wait for “Successfully updated function...”

Give Lambda Permission to Publish to SNS

Step 10: Open Lambda Execution Role

Lambda → Configuration → Permissions

Click the Role name (execution role link)

Step 11: Attach SNS Publish Policy

IAM Role page → Add permissions → Attach policies

Attach: AmazonSNSFullAccess

Click Add permissions

Now Lambda can publish to SNS.

Policy was successfully attached to role.

Creation date

January 06, 2026, 12:08 (UTC+05:30)

Last activity

-

ARN

arn:aws:iam::501168458285:role/service-role/NotificationSimulator-role-idde78md

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (2) Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

☐	Policy name	Type	Attached entities
☐	AmazonSNSFullAccess	AWS managed	1
☐	AWSLambdaBasicExecutionRole-3dea...	Customer managed	1

Test the Mini Project

Step 12: Create a test event

Lambda → Test tab → Create new test event

Event name: PaymentFailed

Paste:

```
{
  "eventType": "PAYMENT_FAILED",
  "user": "Anita"
}
```

Click **Save**

Step 13: Run Test

Click **Test**

Expected:

- Execution: **Succeeded**
- Email should arrive within a few seconds:
Subject like: PAYMENT_FAILED [HIGH]
Message: “Hi Anita, your payment has failed. Please retry.”

PAYMENT_FAILED [HIGH]

Inbox x

AWS Notifications <no-reply@sns.amazonaws.com>
to me

12:16 PM (0 minutes ago) ☆ 😊 ↶ ⋮

Hi Anita, your payment has failed. Please retry.

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.ap-south-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:ap-south-1:501168458285:NotificationTopic:815328c4-95bf-4cbd-8b73-7f95f7f8d84b&Endpoint=amitrlsbca@gmail.com>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

↶ Reply

↷ Forward

😊

Verify CloudWatch Logs

Step 14: View logs

Lambda → Monitor → View CloudWatch logs

Open latest log stream.

Verify these prints exist:

- Notification Simulator invoked
- Event Type:
- Message:
- Priority:

Log management > /aws/lambda/NotificationSimulator > 2026/01/06/[\$LATEST]da21f406118f4e7baf1dbfc1a14320f2

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

1m 1h UTC timezone Display

Timestamp	Message
No older events at this moment. Retry	
2026-01-06T06:46:28.658Z	INIT_START Runtime Version: python:3.11.v109 Runtime Version ARN: arn:aws:lambda:ap-south-1::runtime:49f733259c7ce7e0de...
2026-01-06T06:46:29.089Z	START RequestId: 64253892-a781-4624-9248-a8eed585aeb0 Version: \$LATEST
2026-01-06T06:46:29.090Z	Notification Simulator invoked
2026-01-06T06:46:29.090Z	Event Type: PAYMENT_FAILED
2026-01-06T06:46:29.090Z	Message: Hi Anita, your payment has failed. Please retry.
2026-01-06T06:46:29.090Z	Priority: HIGH
2026-01-06T06:46:29.334Z	END RequestId: 64253892-a781-4624-9248-a8eed585aeb0
2026-01-06T06:46:29.334Z	REPORT RequestId: 64253892-a781-4624-9248-a8eed585aeb0 Duration: 243.59 ms Billed Duration: 672 ms Memory Size: 128 MB
No newer events at this moment. Auto retry paused. Resume	

Expected Outputs

1. Lambda test status: Succeeded
2. Email received from SNS with correct message
3. CloudWatch logs showing event type and generated message

Cleanup

1. Delete Lambda function: NotificationSimulator
2. SNS → delete topic NotificationTopic (subscriptions removed automatically)
3. CloudWatch → delete log group for Lambda
4. (Optional) Delete IAM role if not used elsewhere

DATE: 26-12-25

Exercise - 26: Analyze a CSV File in S3 Using Amazon Athena (No Server)

Upload a small CSV to **S3**, then use **Athena** to run SQL queries like **count**, **average**, **max**, **group by**.

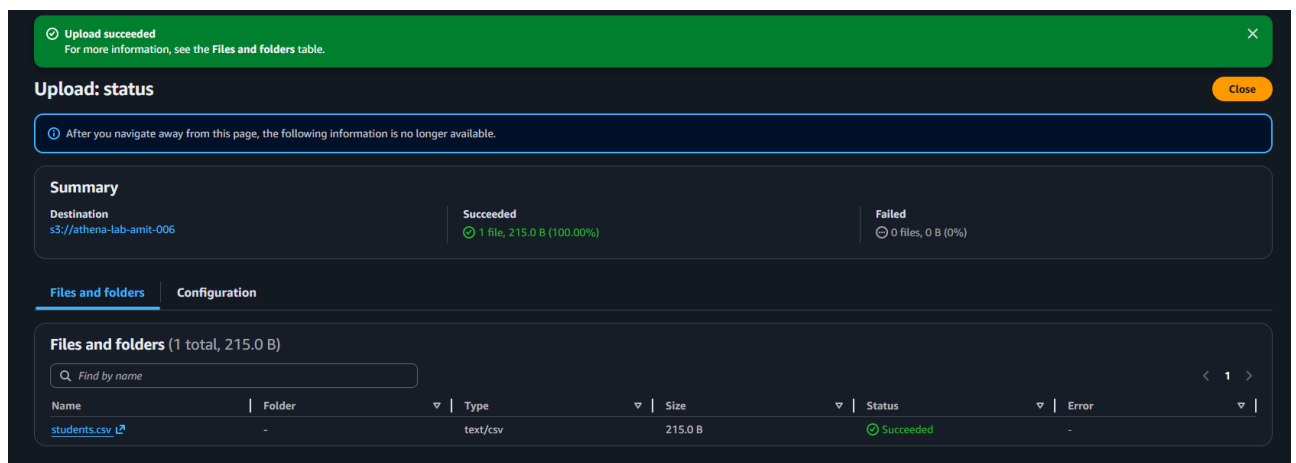
Part A — Create Sample CSV (on your PC)

1. Open **Notepad**
2. Paste this data and save as: **students.csv**

Part B — Create S3 Bucket and Upload CSV

1. Go to **AWS Console → S3**
2. Click **Create bucket**
3. Bucket name: athena-lab-<yourname>-<number> (must be globally unique)
4. Keep defaults → Click **Create bucket**
5. Open the bucket → Click **Upload**
6. Upload **students.csv**
7. Click **Upload**

Now your CSV is in S3.

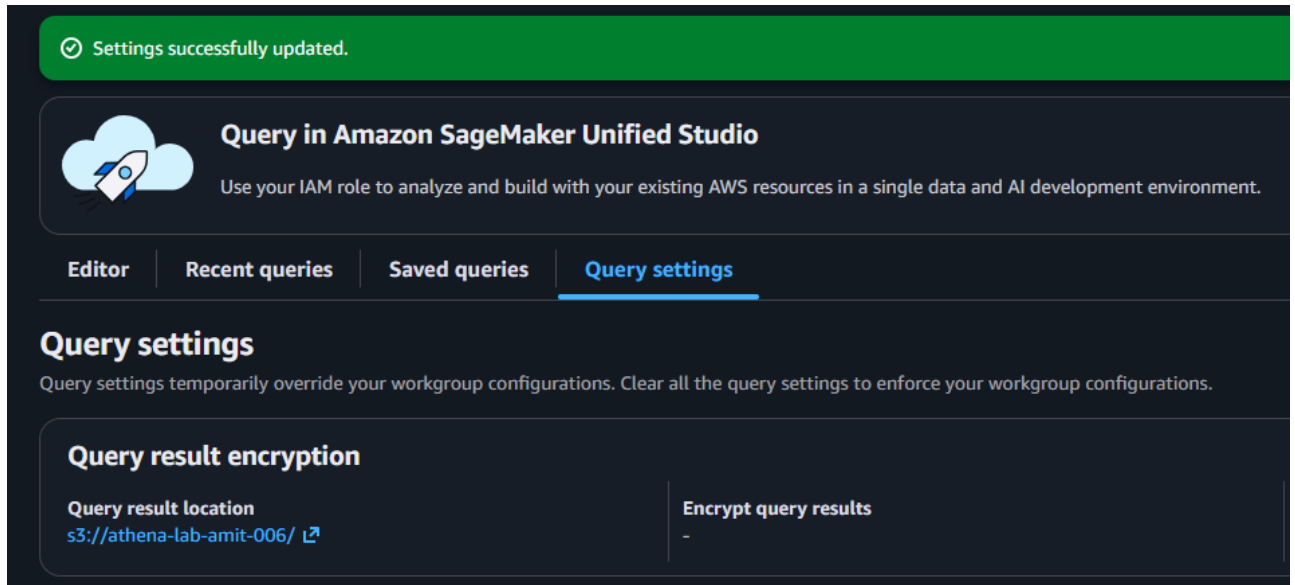


Part C — Open Athena and Set Query Result Location (IMPORTANT)

1. Go to **AWS Console → Amazon Athena**
2. If it shows "Get started" / "Query editor", open it.
3. It will ask to set a **Query result location**
4. Click the link/button like **Settings / Manage / Edit**
5. Set query result location to something like:

- s3://your-bucket-name/athena-results/
6. Click **Save**

This is required so Athena can store query outputs.



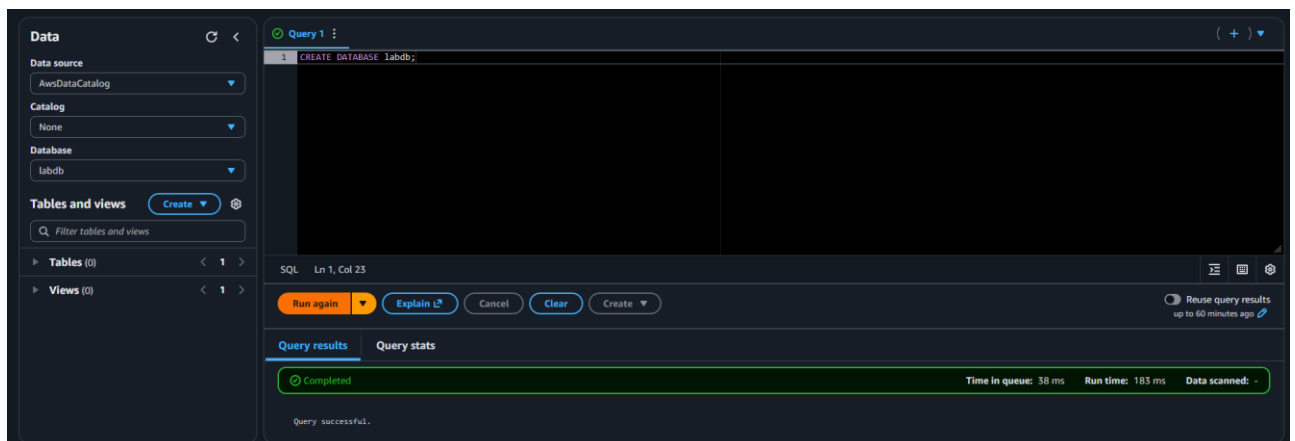
Part D — Create a Database in Athena

In Athena query editor, run:

```
CREATE DATABASE labdb;
```

Then on the left side, select:

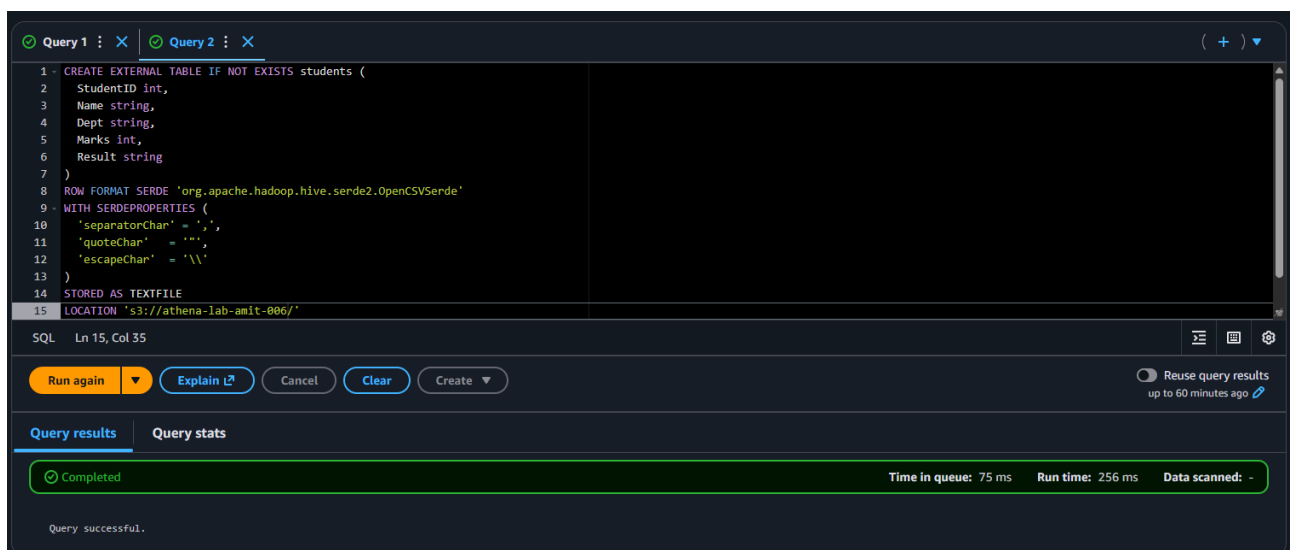
- **Data source:** AwsDataCatalog
- **Database:** labdb



Part E — Create a Table for the CSV in S3

Run this (replace YOUR_BUCKET_NAME with your actual bucket name):


```
CREATE EXTERNAL TABLE IF NOT EXISTS students (  
  
    StudentID int,  
  
    Name string,  
  
    Dept string,  
  
    Marks int,  
  
    Result string  
  
)  
  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
  
WITH SERDEPROPERTIES (  
  
    'separatorChar' = ',',  
  
    'quoteChar' = '"',  
  
    'escapeChar' = '\\'  
  
)  
  
STORED AS TEXTFILE  
  
LOCATION 's3://YOUR_BUCKET_NAME/'  
  
TBLPROPERTIES ('skip.header.line.count'='1');  
  
This tells Athena: “My CSV is in S3, treat it like a table.”
```

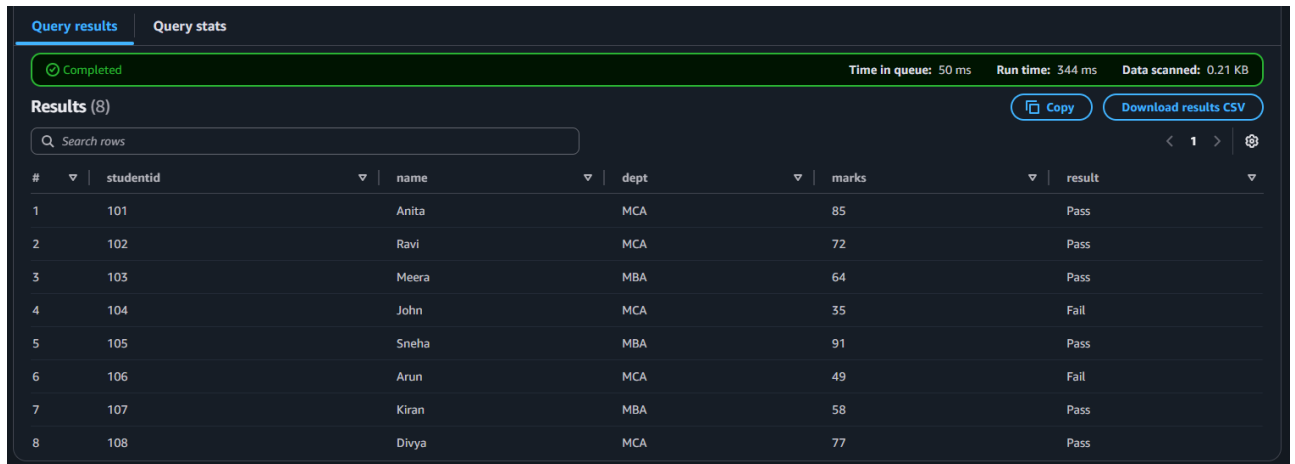


Part F — Run Simple Analysis Queries (Core Part)

1) View all rows

```
SELECT * FROM students;
```

Expected: 8 records.



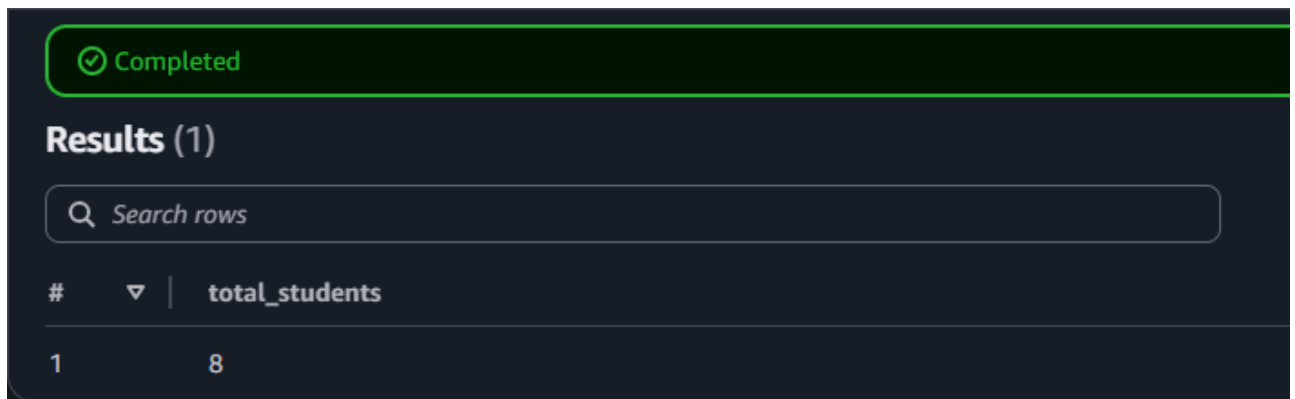
The screenshot shows a database query results interface. At the top, there are tabs for 'Query results' and 'Query stats'. Below the tabs, a green status bar indicates 'Completed' with a checkmark icon. To the right of the status bar, performance metrics are displayed: 'Time in queue: 50 ms', 'Run time: 344 ms', and 'Data scanned: 0.21 KB'. Below the status bar, the text 'Results (8)' is shown. To the right of 'Results (8)', there are two buttons: 'Copy' and 'Download results CSV'. Below these buttons is a search bar with the placeholder text 'Search rows'. Below the search bar is a table with 8 rows and 7 columns. The columns are: '#', 'studentid', 'name', 'dept', 'marks', and 'result'. The rows contain the following data:

#	studentid	name	dept	marks	result
1	101	Anita	MCA	85	Pass
2	102	Ravi	MCA	72	Pass
3	103	Meera	MBA	64	Pass
4	104	John	MCA	35	Fail
5	105	Sneha	MBA	91	Pass
6	106	Arun	MCA	49	Fail
7	107	Kiran	MBA	58	Pass
8	108	Divya	MCA	77	Pass

2) Total students

```
SELECT COUNT(*) AS total_students FROM students;
```

Expected: 8



The screenshot shows a database query results interface. At the top, there is a green status bar with a checkmark icon and the text 'Completed'. Below the status bar, the text 'Results (1)' is shown. Below 'Results (1)' is a search bar with the placeholder text 'Search rows'. Below the search bar is a table with 1 row and 2 columns. The columns are: '#', 'total_students'. The row contains the following data:

#	total_students
1	8

3) Pass vs Fail count

```
SELECT Result, COUNT(*) AS cnt
```

```
FROM students
```

```
GROUP BY Result;
```

Expected (based on data): Pass = 6, Fail = 2

Completed			
Results (2)			
Search rows			
#	Result		cnt
1	Pass		6
2	Fail		2

4) Average marks overall

```
SELECT AVG(Marks) AS avg_marks
FROM students;
```

Query results

Query stats

✔ Completed

Results (1)

🔍 Search rows

#	▼	avg_marks
1		66.375

5) Department-wise average marks

```
SELECT Dept, AVG(Marks) AS avg_marks
FROM students
GROUP BY Dept;
```

Completed			
Results (2)			
Search rows			
#	▼	Dept	▼ avg_marks
1		MCA	63.6
2		MBA	71.0

6) Top scorer

SELECT Name, Dept, Marks

FROM students

ORDER BY Marks DESC

LIMIT 1;

Expected: Sneha (91)

Completed				Time in queue: 64 ms
Results (1)				
Search rows				
#	▼	Name	▼ Dept	▼ Marks
1		Sneha	MBA	91

7) List failed students

SELECT StudentID, Name, Dept, Marks

FROM students

WHERE Result = 'Fail';

Expected: John, Arun

Query results		Query stats	
Completed		Time in queue: 94 ms	Run time: 611 ms
		Data scanned: 0.21 KB	
Results (2)		Copy	Download results CSV
Search rows			
#	StudentID	Name	Dept
1	104	John	MCA
2	106	Arun	MCA

Cleanup (to avoid any charges)

1. In Athena, you can keep DB/table (no cost by itself), but clean S3:
2. Go to **S3 bucket**
3. Delete:
 - students.csv
 - athena-results/ folder contents (query outputs)
4. Delete the bucket (must be empty to delete)

Cost Note (Simple)

- **S3 storage:** tiny (almost negligible for this)
- **Athena:** charges based on **data scanned**; with this tiny CSV it's usually minimal, but **always delete outputs and bucket** after lab.